

UNIVERSITY OF GREENWICH
FACULTY OF ENGINEERING AND SCIENCE
SCHOOL OF COMPUTING AND MATHEMATICAL SCIENCES

Reducing Carbon Footprint of Blockchain Transactions With Optimised Batching System

Zafer Şiar Konyar
001349934

Supervisor: Prof. Georgios Samakovitis

Word count: 14883

COMP1682 Final Year Individual Project
A dissertation submitted in partial fulfilment of the requirements for the degree of
BSc (Hons) Computer Science

April 2026

Abstract

Blockchain technology and cryptocurrencies have become highly popular and widely discussed topics in research, yet its environmental impact remains a critical concern. Although extensive research explores the energy used in mining and trading, the carbon footprint of each individual transaction is not well studied. Existing optimisation solutions focus mainly on network latency and transaction cost. While these metrics are important for user experience, they do not address the ecological footprint of transaction processing. This project proposes an optimised batching system designed specifically to minimise carbon emissions of blockchain transactions. The system is designed to prioritise environmental sustainability as the sole optimisation metric.

To evaluate the proposed system's effectiveness and reproducibility, the system was simulated across two distinct blockchain architectures: Ethereum (Proof of Stake) and VeChain (Proof of Authority). A controlled simulation framework was deployed to measure the gas consumption of individual versus batched transactions, and a rigorous conversion methodology was used to translate this gas usage into quantifiable carbon emissions.

The experimental results demonstrated that the middleware's efficiency depends heavily on the underlying layer-one architecture. On the Ethereum network, the batching system successfully correlated with carbon reduction, achieving a peak carbon emission reduction of approximately 45% at the batch size of 150. Conversely, the middleware failed to reduce the carbon footprint on the VeChain network, consistently recording a carbon emission increase of approximately 20%. This increase was due to the computational overhead required to process cryptographic verifications on-chain, which exceeded the natively low base cost of VeChain's built in multi-task architecture.

The research concludes that while off-chain batching inherently introduces increased transaction latency, it provides a highly viable, carbon reducing solution for non-urgent transactions on networks like Ethereum, where the high fixed base cost of individual transactions significantly outweighs the added computational overhead of verifying aggregated signatures on-chain.



DECLARATION OF AI USE

I have used AI while submitting my assignment in the following ways:

- To develop research questions on the topic – **NO**
- To create an outline of the topic – **NO**
- To explain concepts – **YES**
- To support my use of language – **YES**
- To summarise articles/source(s) – **NO**
- In other ways, as described below – **NO**

Contents

1	Introduction	4
1.1	Problem Domain	4
1.2	Aims and Objectives	5
1.3	Existing Work	6
1.4	Chapter Outline	7
2	Background Research and Project Rationale	8
2.1	Blockchain	8
2.1.1	Ethereum	9
2.1.2	VeChain	9
2.2	Cryptocurrency	10
2.3	Layers of Blockchain	10
2.4	Smart Contracts	11
2.5	Gas	12
2.6	Summary	13
3	Methodology and Technical Approach	14
3.1	Design Goals	14
3.2	Experimental Setup	15
3.3	Evaluation Plan	17
3.4	Evaluation Metrics and Success Criteria	17
3.5	Gas Calculation	18
3.6	Gas-to-Carbon Conversion	18
3.7	Summary	19
4	Implementation	20
4.1	On-chain Layer	21
4.2	Off-chain Layer	23
4.3	Summary	26
5	Results and Evaluation	27
5.1	Experimental Overview	28
5.2	Data Analysis	29
5.3	Ethereum Analysis	29
5.4	VeChain Analysis	32
5.5	Conclusion	34
6	Discussion	35
6.1	Limitations	37
6.2	Future Work	37
A	Smart Contract Source Code	43
A.1	Ethereum Batchers Smart Contract	43

A.2	VeChain Batcher Smart Contract	45
B	Simulation Source Code	47
B.1	Ethereum Simulation Source Code	47
B.2	VeChain Simulation Source Code	52
C	Extended Data Analysis	59
C.1	Ethereum Performance Visualisations	59
C.2	VeChain Performance Visualisations	59

Chapter 1

Introduction

Cryptocurrencies, digital assets of blockchains, have gained global adoption and significantly reshaped economic activity by enabling decentralised value transfer, new financial markets, and alternative models for trust and ownership. Bitcoin, the most widely known cryptocurrency, exhibited a market capitalisation of more than 100 billion USD by 2020 (Sedlmeir et al. 2020). However, blockchain technology is not only the foundation of cryptocurrencies. The main contribution of blockchain technology is to enable trust with a distributed network without a central authority. This revolutionary idea evolved blockchain technology into a fundamental pillar of modern digital infrastructure. It has revolutionised sectors ranging from finance to supply chain management by enabling decentralised and immutable ledgers (Di Pierro 2017).

1.1 Problem Domain

Although cryptocurrency transactions provide secure and decentralised financial networks, their processing mechanisms result in a major environmental cost due to intensive energy consumption. Early consensus mechanisms required immense computational energy, leading to a massive carbon footprint. The process of verifying and adding transactions to a blockchain requires complex computational work, compelling miners to utilise significant processing power. Consequently, a Bitcoin transaction consumes around 1000 kW-hours (kWh) of electricity, which translates to an emission of 700–844 kg of carbon (Bajra et al. 2024, Zhang & Chan 2020).

In 2022, Ethereum underwent a fundamental upgrade known as "The Merge", transitioning its entire network from the energy-intensive Proof of Work to the highly efficient Proof of Stake. While this single event reduced Ethereum's energy consumption by over 99.95% (*Ethereum Energy Consumption* — *ethereum.org* 2025), the carbon footprint of the Ethereum network has not vanished. For instance, despite operating on these more efficient consensus mechanisms, the Ethereum network still produced 2.07 kilotons of carbon emission in 2024 (Cambridge Centre for Alternative Finance n.d.), and the VeChain network generated 12 tons of carbon emission over the last year (VeChainStats n.d.).

This issue is further compounded by the continuous drive towards mass adoption and the scaling of blockchain technology. As blockchain technology expands beyond niche cryptocurrency trading into traditional finance, global supply chain management, and decentralised applications, the sheer volume and throughput of transactions will increase exponentially. Although the industry has implemented solutions to mitigate the carbon footprint of blockchain networks, especially Ethereum's transition to Proof of Stake (PoS), the environmental challenge has not vanished. The environmental concern has moved from the energy intensity of mining to the computational overhead of transaction execution. In non-PoW consensus mechanisms, like PoS, consensus energy is no longer enormous, so the energy consumption of redundant operations (cryptocurrency transactions where every node verifies every transaction) becomes the primary contributor to the total carbon footprint (Sedlmeir et al. 2020). Consequently, as blockchain networks scale to accommodate this higher throughput, the aggregate computational overhead will scale linearly. If global adoption increases by ten or a hundred times, the cumulative energy consumed by

these redundant verifications threatens to offset the environmental efficiency gains achieved by transitioning away from PoW. Therefore, unless the computational overhead of each individual transaction is systematically optimised, the ecological impact of blockchain networks will remain a persistent and growing problem.

Furthermore, there is a significant gap in current transaction optimisation methods, including modern Layer-2 scaling solutions and existing off-chain protocols. Most research and industry implementations focus heavily on network latency and transaction costs. This focus has created a landscape where algorithms are tuned exclusively for financial gain and a seamless user experience. Because these systems are financially driven, they do not hold transactions long enough to maximise the batch capacity. Consequently, they miss the opportunity to significantly dilute the fixed computational base cost across the maximum possible number of transactions. While these financial and speed based metrics are important for commercial viability, environmental sustainability is often neglected. This financially driven approach highlights a distinct lack of research on carbon optimised architecture. To truly address the carbon footprint of transaction processing, a new approach is required that treats increased latency as an acceptable trade-off to maximise batch sizes and achieve optimal environmental efficiency.

1.2 Aims and Objectives

This research addresses this gap by developing an optimised off-chain middleware system that prioritises carbon footprint above all other metrics. To systematically achieve this goal and rigorously evaluate the proposed architecture, the project is structured around four specific and measurable objectives. These include the core system development, the creation of a measurable environmental evaluation metric, a cross-chain comparative analysis, and the algorithmic optimisation needed to balance ecological benefits with practical usability.

Carbon Emission Optimised Middleware Development

The primary aim of this project is to develop a transaction batching middleware system that focuses on environmental sustainability. This research seeks to quantify exactly how much carbon emission can be reduced with the proposed optimisation system by shifting the optimisation metric from financial cost to carbon footprint.

The proposed middleware employs an aggregation method called batching. Rather than processing every request individually on energy-intensive Layer-1, the proposed middleware collects all requests off-chain and batches them into a single transaction. By doing so, the fixed base cost is effectively shared across all transactions within the batch, reducing the cost per transaction. This approach minimises the number of on-chain operations, ensuring that the carbon footprint of the network is kept to an absolute minimum.

Developing a Gas-to-Carbon Conversion Metric

Every transaction submitted to the blockchain network must be verified by its nodes, which requires computational power. To account for this cost, blockchain networks charge users based on the amount of computational power their transactions consume, using a metric called gas. Gas represents the computational effort required to execute operations such as transaction validation and state updates. As an example, for every transaction, users are charged a fixed base cost of 21,000 gas units on the Ethereum network and 5,000 gas units on the VeChain network. As transactions get more complex, particularly those that involve smart contracts, require higher amounts of gas due to additional computational steps involved. This pricing mechanism helps ensure that network resources are used efficiently.

This research also aims to develop a gas-to-carbon transition metric to evaluate the performance of the proposed middleware and quantify the carbon emission reduction. Although gas efficiency measures computational effort, it remains an abstract network metric. To assess environmental sustainability, this

metric must be translated into a tangible ecological footprint. Therefore, the project will establish a rigorous conversion methodology that uses network data to calculate carbon emissions per unit of gas. This metric will serve as the primary evaluation tool, allowing for a direct and measurable comparison between the carbon footprint of legacy individual transactions and the proposed batched executions.

Cross-Chain Network Analysis

Another key objective of this project is to analyse the proposed middleware on different blockchain network architectures. The proposed batching system is not evaluated on a single blockchain network alone. Instead, it will be simulated and tested across two blockchain networks that adopt different consensus mechanisms, Ethereum (Proof of Stake) and VeChain (Proof of Authority), to observe how different consensus mechanisms and network architectures influence the proposed system. To ensure experimental consistency and represent a realistic usage scenario across these distinct architectures, the system's performance will be evaluated through simulated USDC transfers.

Parameter Optimisation

Finally, a critical objective of this research is to determine the optimal configuration of the proposed middleware. Because off-chain batching inherently introduces transaction delays, the system must balance environmental efficiency with practical usability to prevent infinite transaction delays to reduce carbon emissions. This project will systematically evaluate and manipulate the system parameters to identify the precise configuration that maximises carbon emission reduction while keeping the system usable.

1.3 Existing Work

Batching has been used by most companies to reduce the cost of transactions. Most of the work about optimising the batching process focuses on reducing the cost and latency of cryptocurrency transactions. However, the solutions provided can be altered to reduce the carbon footprint as well.

According to Wang et al. (2021), batching of transactions should be performed off-chain to minimize the execution cost. Smart contracts run on the blockchain network, which makes them computationally expensive. The suggestion is to collect all transactions and batch them off-chain, then send the batched transactions to a smart contract to execute, which would reduce the cost. Wang et al. (2021) introduce three policies for optimising:

1. **Policy W sec:** Batch every transaction that arrives in W seconds.
2. **Policy Top 1:** Batch all transactions that come from the most active wallet.
3. **Policy Min X :** Batch only when there is a minimum of X transactions in a batch time window.

To evaluate these policies, they merged the "Min X " and "Top 1" policies with the W sec policy. By setting the time frame to 120 seconds and the minimum batch threshold to 5, they formulated two test cases, 120sec-min5 and 120sec-top1. Their findings indicate that both methods result in a reduction in cost. However, the 120sec-min5 method has more impact on reducing gas costs than the 120sec-top1 method. This occurs because the min5 policy batches all of the invocations, whereas the top1 policy batches transactions from the single most active invocation type, which are trade invocations. Consequently, the min5 policy generates batches that are 125% larger than those generated by the top1 policy. This caused the min5 policy to have approximately 24% gas reduction per invocation, whereas the top1 policy caused 15% gas reduction. They also applied a min policy to the BNB token, which resulted in 59% gas reduction compared to the unbatched baseline (Wang et al. 2021).

Jia et al. (2024) also suggest that the organising and batching processes should be performed off-chain. They introduce the accelerator process, which is a middleware to collect all the transaction data

from every user, organise and batch the transactions, and send all the batches to the network. They also introduce Lyapunov Optimisation Theory to optimise the batching process.

Lyapunov optimisation refers to the use of the Lyapunov function to make a dynamic system optimal. It is widely used in queuing systems such as batching. The main idea is the drift-plus-penalty expression. Drift means the Lyapunov function changes over time and the penalty represents the system objective. This formulation helps systems balance stability and performance (Chang et al. 2023).

Wang et al. (2020) created an off-chain middleware system to synchronise data from centralised databases to a blockchain efficiently and consistently. They sent the data to the blockchain network by converting SQL queries into a key-value pair format and submitting multiple pairs together in a single batched transaction. To test the batching performance, Wang et al. (2020) incrementally increased the batch size from 200 to 1000. As the batch size increased, the throughput increased as well until it reached the threshold (around 400) and remained unchanged above that threshold. However, as the batch size increased, the latency also increased because the individual transaction waited longer to be included in a batch. The results suggest that too small batch size results in poor performance, whereas too large batch size results in high latency.

1.4 Chapter Outline

This report is structured into six chapters detailing the research, implementation, and evaluation of the proposed batching system. Following this introductory chapter, Chapter 2 establishes the foundational concepts critical to the project, including blockchain architecture, smart contracts, and gas consumption mechanisms. Chapter 3 details the methodology and technical approach, outlining the simulation framework, evaluation metrics, and the mathematical model used to convert gas usage into measurable carbon footprint. Next, Chapter 4 explains the implementation of the project's subsystems: On-chain and Off-chain layers. Chapter 5 presents the results and evaluation, providing rigorous analysis of the simulations across both the Ethereum and VeChain networks to assess carbon emission reductions and latency trade-offs. Finally, Chapter 6 explores the broader implications of the results, highlights the limitations of the isolated simulation environment, and outlines directions for future work.

Chapter 2

Background Research and Project Rationale

Before detailing the rationale and technical architecture of the proposed system, it is essential to establish a clear understanding of the foundational technologies involved. This chapter explains the core ideas of blockchain in a clear progression, starting from the basic mechanics of how it works and its different consensus protocols, and moving toward the use of cryptocurrencies and stablecoins such as USDC. It describes the modular structure of blockchain systems, especially the difference between Layer-1 networks and off-chain middleware, to show why moving computation away from energy-intensive environments can be beneficial. It also highlights the role of smart contracts as self-executing programs that handle transactions, along with the gas mechanism used to measure the computational effort required. Together, these concepts help explain how transaction complexity and network designs, such as Ethereum’s Proof of Stake and VeChain’s Proof of Authority, can affect the carbon footprint of blockchain systems. Understanding these relationships is important for explaining why the proposed batching system focuses on reducing specific computational overheads to improve environmental sustainability.

2.1 Blockchain

Blockchain is a ledger shared among all the users in a network. New recordings are constantly added to this network and form blocks. Blockchain consists of multiple blocks linked together forming a chain, and each block contains a hash pointer to the previous block to form the chain (Hashemi Joo et al. 2019).

Blockchains use cryptography to protect the network from being tampered with. Everybody in the network has crypto wallets, and these wallets have two keys: a public key and a private key. The public key is similar to the bank account number in the centralised banking system. It is the address of the crypto wallet. Users use it to receive transactions from other users in the network. The private key is similar to a password. It gives the user the authority to send transactions to other users using their crypto wallets. All transaction information is saved in the blocks on the blockchain. Once recorded, records in blocks cannot be altered without the collusion of the majority of the network (Zohuri et al. 2022, Hashemi Joo et al. 2019).

Blockchains use fault-tolerant protocols called consensus mechanisms to preserve agreement among the nodes in the network. Consensus mechanisms ensure that as the network expands, all participants can reach a single decision on the authenticity of transactions. Proof of Work (PoW), Proof of Stake (PoS), and Proof of Authority (PoA) are examples of consensus mechanisms (Lashkari & Musilek 2021). Although the blockchain ecosystem consists of numerous networks with diverse protocols, this research narrows its experimental focus specifically to Ethereum and VeChain. To properly evaluate how the proposed batching middleware mitigates carbon emissions across different infrastructures, it is essential to establish a deeper understanding of these two networks, as their distinct consensus mechanisms and architectural standards serve as the primary variables for the cross-chain comparative analysis.

PoW Proof of Work forces a complicated puzzle to be solved by cooperating nodes to maintain agreement across the network. Miners solve the puzzle by estimating a specific value known as nonce to be granted access to include a new block (Lashkari & Musilek 2021).

PoS Proof of Stake is a mechanism that ensures validators commit something of value to the network, which can be lost if they act dishonestly. In Ethereum's implementation, validators stake ETH into a smart contract, which gives them the responsibility of validating new blocks and occasionally proposing them. If they try to defraud the network (for example, by proposing multiple blocks when they ought to send one or sending conflicting attestations), some or all of their staked ETH can be destroyed. Initially proposed for Peercoin to reduce power consumption, Proof of Stake eliminates the need for computational mining by determining block proposers based on stake size rather than account balance. Ethereum fully transitioned to this model in 2022 through "The Merge", an upgrade that eliminated energy-intensive mining and reduced the network's energy consumption by over 99.95%. However, with the consensus energy drastically lowered, the energy consumed by redundant operations, where every node verifies every transaction, has now become the primary contributor to the network's remaining carbon footprint (Lashkari & Musilek 2021, ethereum.org 2025).

PoA Proof of Authority is a consensus algorithm for permissioned blockchains. PoA does not allow anonymous block producers; instead, there is a fixed number of known validators. These known validators, also known as Authority Masternodes, are responsible for keeping the network safe. PoA operates within time divisions where these Authority Masternodes propose blocks using a round-robin rotation. A block must be signed by a majority of these authorised entities to be accepted. It achieves better performance by employing a significantly lighter message transmission scheme (Lashkari & Musilek 2021, VeChain Foundation 2024).

2.1.1 Ethereum

The first blockchain, called "Bitcoin", was implemented into practice by Satoshi Nakamoto in 2009. Satoshi's main intent for Bitcoin was for it to be used as a digital currency without any control of a centralised authority.

Following the success of Bitcoin, Ethereum was introduced by Vitalik Buterin in 2014 and took its place to be the second largest blockchain network in the world. Buterin's intent with Ethereum was to allow application developers to create applications that benefit from the decentralisation and security features of the blockchain with the power of a Turing-complete virtual machine, the Ethereum Virtual Machine (EVM) (Buterin 2016). The main feature Ethereum introduced was smart contracts [2.4] which are programs that run on the Ethereum network. The Ethereum Virtual Machine (EVM) serves as a crucial runtime environment for smart contracts on the Ethereum network. The EVM compiles the contract's high-level source code into a specific series of opcodes. Consequently, every node on the Ethereum network requires an EVM to successfully validate transactions and execute smart contracts (Ma et al. 2021).

For many years, Ethereum used Proof of Work as its consensus mechanism, but in 2022, Ethereum transitioned its consensus mechanism to Proof of Stake. This shift, named "The Merge", eliminated the need for energy-intensive mining and reduced the energy consumption.

2.1.2 VeChain

VeChain was first introduced in 2017. The main purpose of VeChain is to provide businesses with a secure, decentralised architecture that allows for fast value transfers, transparent information flow, and seamless collaboration by offering a proper governance model, a sustainable economic model, and a flexible blockchain (*Web3 for Better Whitepaper 3.0* 2023).

The blockchain network used by VeChain is named VeChainThor. VeChainThor is an open-source public blockchain network that uses Proof of Authority as its consensus mechanism, where transaction

validation and platform evolution are governed through a structured authority system with defined governance and voting processes. There are two tokens that are embedded within the VeChainThor network, the VeChain token (VET) and VeThor (VTHO). VET acts as the primary store of value, and VTHO represents the mandatory payment token for network processing energy (to cover gas fees [2.5]) (Petrović et al. 2022, *Web3 for Better Whitepaper 3.0* 2023).

2.2 Cryptocurrency

One of the most prominent applications of blockchain is cryptocurrencies. Cryptocurrency is a virtual currency that uses encryption techniques to regulate the creation of additional value units, verify the transfer of funds, and operate without a centralised authority. It utilises distributed ledger technology to enable secure transactions through a peer-to-peer mechanism that effectively eliminates the "middle-man", such as a financial institution. Cryptocurrencies are typically stored in "crypto wallets". Wallets are similar to bank accounts in centralised finance, but they are designed to preserve privacy and do not collect any personal information about wallet owners. Although these represent the native currencies of blockchain networks, blockchains also facilitate other digital assets called tokens (Härdle et al. 2020, Hossain 2021).

Tokens

Tokens are sub-currencies that are built on top of a blockchain via smart contracts that can potentially include features that native currencies do not have. They are also considered digital assets. However, they do not represent the native asset of the blockchain that they are built on; they rely on the native coin's infrastructure (Buterin et al. 2014). Tokens facilitate value transfer as a medium of exchange based on a unit of account. They can represent various assets such as tangible items like real estate and gold, or intangible rights like voting rights and ownership. Tokens also provide economic and commercial benefits by reducing transaction costs, easing access and market participation, and improving market liquidity (Marin et al. 2023).

Tokens are essentially programmable smart contracts that grant developers significant creative freedom. However, the resulting diversity in rules and features resulted in interoperability issues between applications and made them harder to interpret. To solve this, blockchain networks implement a set of standards to provide a framework for token deployment and functionality, such as ERC for Ethereum and VIP for VeChain. While tokens can represent various assets, one of the most common uses of tokens are stablecoins (Marin et al. 2023).

Stablecoins Stablecoins can be defined as digital assets designed to maintain a stable valuation relative to a specific benchmark, typically a fiat currency. They control exchange rates between tokens and fiat currencies by implementing stabilisation mechanisms. Stablecoins are backed by traditional assets; for instance, Tether (USDT) is a stablecoin that is backed by USD and intended to maintain parity with the US Dollar. They also provide value preservation, peer-to-peer liquidity, and standardised financial reporting across the network (Makarov et al. 2022, Hassani et al. 2018, Mita et al. 2019).

2.3 Layers of Blockchain

The blockchain architecture consist of three fundamental layers, forming a modular structure. These layers can be distinguished as: layer-zero, layer-one and layer-two.

Layer-Zero Layer-zero, also known as "The Network Layer", is a peer-to-peer layer where blockchain nodes exchange information asynchronously; it provides robust communication between two nodes. Layer-zero plays an important role in determining the scalability, security, and privacy of blockchain systems (Gudgeon et al. 2020).

Layer-One Fundamentally, layer-one is the core blockchain itself. It maintains an immutable, append-only ledger that aggregates network transactions for public verifiability. Depending on its scripting language, layer-one supports either simple asset transfers or complex applications via smart contracts [2.4]. Layer-two protocols rely on this base layer to correctly record and finalise data (Gudgeon et al. 2020).

Layer-Two Layer-two allows transactions between users by serving as a medium on top of layer-one and links the transactions to the blockchain network (Rebello et al. 2024, Gudgeon et al. 2020). There are also blockchains that are built on layer-two that inherit data availability and security properties from the layer-one blockchain they are built on top of by deploying a set of smart contracts on the layer-one blockchain (Zhao & Luo 2025).

Middleware is similar to layer-two blockchains. However, layer-two blockchains are positioned on top of layer-one, inheriting properties such as security and data immutability from layer-one. Whereas middleware is positioned in between the user and the blockchain network to provide services the blockchain cannot provide natively or efficiently.

2.4 Smart Contracts

The smart contract concept was first proposed by Nick Szabo in 1994. A smart contract is a self-executing computer protocol embedded in the blockchain network that has self-verifying, self-executing, tamper-resistant properties, and it consists of the value, address, functions and state. The purpose of a smart contract is to take a transaction as an input, execute its logic code, and trigger the output events without third-party involvement (Mohanta et al. 2018).

How Transactions Happen with Smart Contracts?

There are two parties included in a cryptocurrency transaction: a sender and a receiver. First, the sender creates a transaction containing two inputs — the transaction amount with the receiver’s public key and the smart contract address — and signs it with their private key for authentication. Then, this transaction is submitted to the blockchain for validating and executing it according to the smart contract’s predefined rules and logic. Once the execution finishes, the network sends the specified amount of cryptocurrency to the receiver’s wallet and records the transaction to a block in the network.

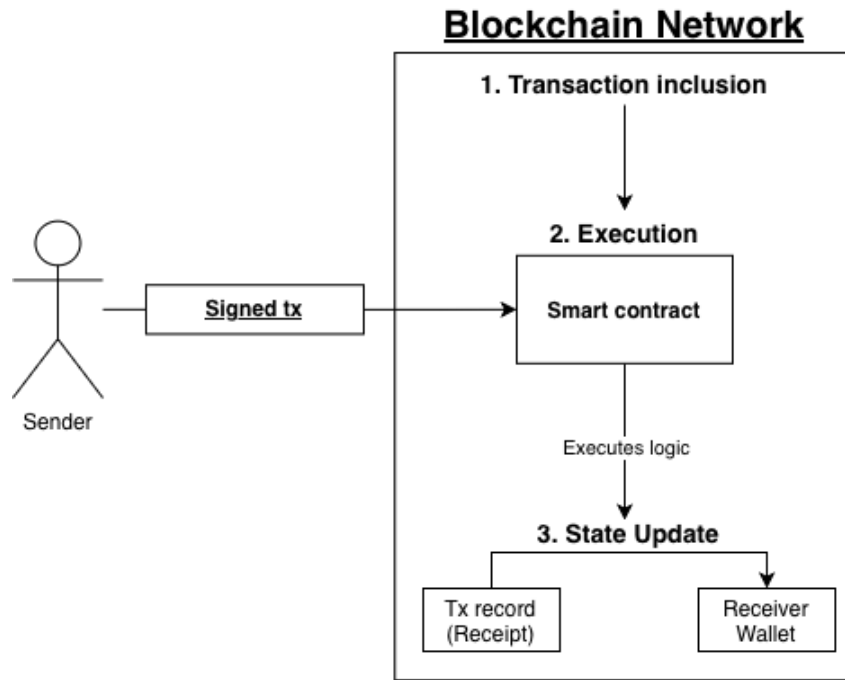


Figure 2.1: Basic visualisation of transaction process (Created by the author).

2.5 Gas

As mentioned before, every transaction that occurs in Ethereum Virtual Machine (EVM) uses computational power. Gas is a virtual measurement unit to determine the computational cost of transactions on the EVM. The purpose of it is to convert computational costs to payable transaction fees (Yang et al. 2019). Miners offer their computational resources to end-users enabling the deployment and execution of smart contracts, and in return, they take Ethers which is calculated by the total gas consumed. Therefore, users need to pay the gas fees for their transactions to be processed, as these fees compensate for the computational and validation work required to execute and record the transaction on the blockchain (Khan et al. 2022).

While users typically pay gas fees directly, alternative models have emerged to shift this financial burden away from the user, introducing the concept of gasless transactions.

When a transaction is processed in the blockchain network, the user sets a gas limit. To ensure validators are compensated for verification costs, the gas limit is set to the maximum price the caller is willing to pay to have their transaction processed. Without the gas limit, some faulty smart contracts can stuck into an infinite loop that causes the network to freeze. Or if the user sets the gas limit below the actual execution cost of the task they want to execute, the task fails and the user will not get refunded (Ta & Do 2024).

Smart contracts also cost gas to be deployed and interacted with since they also use computational power. They include a number of logic and tasks to execute; that is why the gas cost varies for every smart contract. Essentially, the more complex the code is and the more data it needs to store or modify on the blockchain network, the more computational power it is going to require to be processed. Porkodi & Kesavaraja (2024) demonstrates strategies for optimising smart contracts. These methodologies will be used to reduce gas consumption within this project.

Gasless Transactions

The term "gasless transaction" does not mean a type of transaction. Instead, it is a method to shift the financial burden by using a third-party that pays the gas cost for the users, meaning that while the process

appears 'gasless' from the user's perspective, it is not technically gasless for the network.

For example, Sheffield & Wijeyekoon (2024) introduces a third-party fee abstraction mechanism called "Portal" that enables users to pay the gas fees with the currency they are transferring instead of the native currency of the blockchain network. And Hui (2023) introduces LightLink, which achieves gasless transactions through a subscription-based model called "Enterprise Mode" rather than individual fee abstraction. In this system, enterprises pay a flat fee to whitelist their smart contracts, effectively subsidising the gas costs for their users.

Madhwal et al. (2025) also mentions a similar gasless transaction model designed for autonomous IoT devices in which a third-party mechanism pays the gas fee on behalf of the user. However, their experimental results prove that this reliance on using a third-party gas fee payer can come at a steep environmental cost. They demonstrated that the additional computational overhead increased gas consumption by approximately 82% compared to standard transactions, directly increasing the carbon footprint.

2.6 Summary

This chapter established the technical foundation of the research by examining blockchain architectures, the execution of smart contracts, and the mechanics of gas consumption across both the Ethereum and VeChain networks. By identifying the limitations of current transaction models and the specific role of stablecoins like USDC, the project rationale for a carbon-optimised middleware was solidified. Having defined these core principles, the following chapter will detail the methodology and technical framework, including the design goals and the gas-to-carbon conversion model, required to implement and evaluate the proposed batching system.

Chapter 3

Methodology and Technical Approach

This project proposes the design and implementation of an off-chain transaction batching middleware that focuses on reducing the carbon footprint. The layer-one network is highly energy-intensive due to the computational workload to maintain the decentralised consensus and state mechanism. This energy cost is incurred for all transactions and computational tasks processed within smart contracts, leading to significant redundancy when handling high throughput. To mitigate this, the proposed system shifts the transaction batching process off-chain, where computational overhead is significantly lower. The mechanism batches a queue of incoming transactions and aggregates them into a single commit. This process drastically reduces the number of required on-chain executions, ensuring that the energy-intensive layer-one network is triggered only once to execute multiple transactions rather than triggering for each individual transaction (Wang et al. 2021, Jia et al. 2024).

This project adopts a pattern to prioritise environmental sustainability. To achieve that, this project explicitly accepts trade-offs, decentralisation and latency. By implementing an off-chain batcher to organise the transactions before they reach layer-one, the system reduces the computational energy requirement for transaction processing. This architectural choice reduces decentralisation by introducing a centralised point of coordination for transaction management. Furthermore, this system also introduces latency, as individual transactions must be held to be included and executed in a batch. However, completely disregarding latency and security concerns is not realistic for an interactive system. Consequently, latency is de-prioritised in favour of environmental sustainability, functioning as a soft constraint to ensure basic usability.

3.1 Design Goals

This project's design is driven to accomplish specific objectives aimed at finding an optimised approach to minimise the carbon footprint of crypto transactions. The primary goals governing the system architecture are efficiency quantification, parameter optimisation, cross-chain consensus compatibility, and scalability assessment.

Efficiency Quantification The primary goal of this project is to quantify the energy efficiency of the proposed system relative to the current industry benchmarks. The objective is not merely to claim a reduction, but to mathematically demonstrate the delta in energy consumption between the proposed transaction architecture and legacy transaction implementations. For the proposition to be technically viable, the proposed system should demonstrate a statistically significant reduction in the carbon footprint. The observed results of this project will determine the feasibility of the proposed model. The project will evaluate whether the magnitude of the carbon reduction justifies the trade-offs, such as centralisation and latency.

Parameter Optimisation This project also aims to identify the most suitable values for each variable that is going to be used in the proposed system to achieve maximum carbon reduction. To achieve this

goal, it is required to treat the system as a multi-variable optimisation problem that prioritises carbon reduction. The variables that are going to be used in the proposed architecture will be systematically adjusted to observe their impact on the system’s energy efficiency. To build a usable system, security and latency concerns will be taken into account as well; however, the priority weights of these metrics will be minimal to accomplish the highest efficiency goal. This ensures that the system to reach the peak energy efficiency without falling behind the minimum usability thresholds. The optimisation process seeks to find the ideal requirements of the system variables to determine the optimal carbon reduction and efficient ratio.

Cross-Chain Consensus Compatibility To evaluate the reproducibility and adaptability of the system across different network environments, the system will be executed on Ethereum and VeChain networks. This comparison is crucial as it subjects the off-chain algorithm to two distinct consensus mechanisms, PoS and PoA. By verifying the model on both networks, this project aims to prove that its energy efficiency is independent of the network protocol rather than being optimised for a single environment. This demonstrates that the proposed architecture can serve as an efficient transaction method for other existing blockchain networks, regardless of their underlying consensus mechanism.

Scalability Assessment Another aim of this project is to measure system efficiency under different throughput. Although blockchain systems offer scalability for a high volume of transactions, when they are often compared to legacy payment systems like VISA, blockchain networks demonstrate significantly low transaction throughput, as measured by transactions per second. According to etherscan.io (2015), the Ethereum network demonstrates an average transaction throughput of roughly 25 transactions per second as of 2026, whereas conventional payment systems like VISA can process approximately 2,000 transactions per second (Bez et al. 2019). This project aims to simulate the proposed system under average, high, and low throughput conditions. Examining the system across different throughput levels enables the evaluation of the robustness of the proposed architecture under both congested and uncongested network states.

3.2 Experimental Setup

To examine the carbon reduction of the proposed system, a controlled simulation framework is developed. This framework isolates the transaction execution, allowing for a direct comparison between individual and batched transaction propagation. The experiments are conducted on two distinct networks, Ethereum and VeChain Thor, utilising USDC as the transfer asset.

USDC is a fiat-backed stablecoin pegged 1:1 to the USD. As of 2026, USDC has the second biggest stablecoin market share with 26% (US\$75.2B), which rose from 22% in the last year. Consequently, simulating USDC transfers ensures the results reflect a realistic usage scenario. On Ethereum, USDC operates as a standard ERC-20 smart contract. On the VeChain network the equivalent token standard is VIP-180, which is designed as a functional superset of ERC-20 to ensure compatibility, which means the computational logic for transferring USDC is nearly identical on both networks. By maintaining a consistent token logic, the simulation framework ensures the system’s viability across both PoS and PoA consensus mechanisms (Binance Research 2026).

After executing simulations, gas consumption data from individual and batched transactions will be collected from transaction receipts. This data will then be converted to carbon emissions using the defined gas-to-carbon conversion model mentioned in Subsection 3.6.

The resulting dataset will be used to quantify the percentage reduction in carbon emissions achieved through the proposed batching system across both networks. Then, statistical analysis will be conducted to evaluate the influence of independent variables on carbon emissions. The findings will guide parameter optimisation to determine the configuration that minimises carbon emissions while maintaining system usability.

Experimental Variables

The experimental variables used in the framework are divided into dependent and independent variables. The independent variables are manipulated in each simulation run to observe their correlation with carbon reduction and to identify the optimal parameter configuration that maximises environmental efficiency.

Independent Variables

- **Batch Size (N):** The number of token transfers batched into a single transaction. Batch size is manipulated to determine the optimal ratio between batched transaction count and gas reduction. In both Ethereum and VeChain, a portion of the gas cost is the base cost required to execute the transaction. The base cost is 21,000 for Ethereum, 5,000 for VeChain (Wood et al. 2014, VeChain Foundation 2025b). By increasing N , the system amortises this fixed gas cost across multiple transfers, directly lowering the carbon footprint per transaction.
- **Batch Interval (t):** The maximum duration for transaction accumulation. The system executes the batch at the end of time t if the target batch size N is not reached. The batch interval is included to prevent infinite wait times during periods of low throughput in order to keep the proposed system usable.
- **Network Throughput (λ):** The rate of transactions submitted to the system every second. This variable is used to simulate average, high, and low traffic conditions to test the system's robustness under different congestion levels. Simulating different levels of throughput allows the study to examine if the batching system remains carbon efficient even when the network congestion changes.
- **Network:** The simulations will run on Ethereum (PoS) and VeChain (PoA) blockchain networks. This variable is chosen to prove cross-chain consensus compatibility. By testing the system on both Ethereum and VeChain, the results can determine if the proposed system is effective across different consensus mechanisms and blockchain architectures.

Dependent Variables

- **Gas Consumption (G_{total}):** The total computational unit of both individual and batched transactions. It serves as the primary foundational metric required to calculate the carbon emissions of the system. Tracking gas consumption is essential, as the project's success relies on demonstrating a significant reduction in gas usage.
- **Carbon Footprint (CF_{CO_2}):** The total carbon emissions of the system, and it serves as the key performance indicator for the system. It is included to shift the narrative from financial cost to environmental impact, fulfilling the project's primary rationale. The carbon footprint is calculated by using the gas consumption (G_{total}).
- **Latency (L):** The time delay that transactions have to wait before the batch is executed. While deprioritised in favour of environmental sustainability, it is tracked to ensure the system maintains basic usability.

Simulation Framework

The simulation framework is designed as a controlled, isolated environment that interfaces with the private blockchain networks. The simulation consist of two main layers, the blockchain network and the off-chain middleware. The core component of the system is the off-chain middleware that acts as the coordinator to collect transactions into batches and sends them to the blockchain network for execution.

To ensure measurement accuracy, the framework employs a dual-path execution model. To evaluate the system’s robustness, the middleware generates synthetic transaction loads at varying throughput levels, representing low, average, and high network congestion. Every token transfer submitted to the off-chain middleware is processed individually under standard execution conditions. The same transactions are then routed through the batching system, enabling a direct comparison between individual and batched transactions.

To isolate the network and the consensus mechanism as the primary variables for cross-chain comparison, the simulation uses USDC as the transfer asset for both networks. By using a stablecoin with nearly identical computational logic, the simulation framework ensures that the observed results are primarily affected by the batching system and the underlying consensus mechanisms, rather than differences in token smart contract complexity.

3.3 Evaluation Plan

The evaluation of the proposed system is going to be based on a structured plan focusing on research quality, technical validity, and environmental impact. The objective is to determine whether the proposed batching system has a significant impact on the gas usage of both Ethereum and VeChain token transactions. The evaluation of this project extends beyond measuring the gas used by transactions by converting gas to carbon consumption to indicate the significance of the project in terms of environmental impact. After executing, used gas information will be collected for both individual and batched transactions from transaction receipts, to convert gas used into carbon emissions of the transactions. The significance will be measured based on the significance value $p < 0.05$.

3.4 Evaluation Metrics and Success Criteria

To systematically assess the viability and effectiveness of the proposed system, the evaluation is categorised into four key aspects: gas efficiency, carbon footprint estimate, performance stability, and research quality. The fundamental success of this project relies on demonstrating a significant reduction in gas usage. This directly translates into a proportional decrease in carbon emissions, which is the primary objective of this project. Beyond environmental efficiency, the system must exhibit performance stability through statistically consistent results across multiple simulations. Finally, the project is held to strict research quality standards, requiring all methodologies, code, and results to be transparent, well documented, and fully reproducible on GitHub.

Aspect	Method	Success Criteria
Gas Efficiency	Measure gas used for each transaction for both individual and batched executions	Batched transaction shows a significant gas usage reduction compared to individual transactions
Carbon Footprint Estimate	Convert gas usage to carbon footprint using credible literature	Estimated emissions decrease proportionally with reduced gas consumption
Performance Stability	Record variance and repeatability across multiple simulations on the network	Results are statistically consistent across simulations
Research Quality	Evaluate clarity, transparency, and reproducibility of batching system and documentation	All results, code, and methodology are well documented and reproducible on GitHub

3.5 Gas Calculation

The total gas used in a regular Ethereum transaction is calculated by $g_{base} + g_{execution}$. The base cost, g_{base} , is the base cost of every transaction, which is equivalent to 21,000 gas units. The execution cost, $g_{execution}$, is the gas used to execute the transaction, which varies for every transaction according to its computational complexity (Wood et al. 2014). However, batched transactions have only one base cost, so even without optimising, if a batch has n transactions inside of it, it would save $(n - 1) \times g_{base}$ gas units.

On the VeChain network, gas calculation is divided between the transaction container and its initial operations, known as clauses. The total gas cost is calculated by $g_{total} = g_0 + \sum(g_{clause})$, where g_0 is the intrinsic base cost, a fixed overhead of 5,000 units, to initiate the transaction. The remaining cost is the sum of gas used for each clause. g_{clause} has three variables inside: type (g_{type} , 16,000 for regular transactions), data payload (g_{data}), and virtual machine execution (g_{vm}). Because the protocol supports multi-task transactions, a single transaction can execute n clauses while incurring g_0 once, which results in saving $(n - 1) \times g_0$ gas compared to sending n individual transactions (VeChain Foundation 2025b).

In this project, the execution gas cost of each transaction will be traced by using transaction receipts for each network. Each simulation will calculate the total gas used by the individual transactions and the batched transactions.

3.6 Gas-to-Carbon Conversion

To translate gas results into carbon emissions, a special conversion methodology has been developed for this project. The approach ensures that the carbon emission estimates achieve maximum precision by using the most recent data. Furthermore, for the Ethereum network specifically, the project also uses an API provided by Digiconomist (Digiconomist n.d.). This API provides a metric for the carbon footprint per unit of gas in grams. This serves as a secondary method to double-check the efficiency of the model.

Primary Conversion Method

After calculating the used gas of transactions, to convert the gas used into carbon footprint, this project uses a detailed conversion model.

To evaluate the environmental impact, this project utilises the daily carbon emissions of the Ethereum network sourced directly from the Cambridge Centre for Alternative Finance (Cambridge Centre for Alternative Finance n.d.).

The other metric used in this method is the daily gas used for Ethereum, which can be obtained from etherscan.io (2015), a block explorer and analytics platform widely regarded as the primary data source for the network.

The proposed method for VeChain shares the same methodology as that of Ethereum, relying on daily carbon emissions and daily gas usage. VeChain provides all the necessary data on VeChainStats (n.d.). VeChain Stats is a block explorer, similar to etherscan.io, that is powered directly by VeChain, ensuring high accuracy for the analysis.

After gathering the necessary data, the estimated carbon footprint of a single gas unit calculated using the formula below.

$$CE_{\text{unit-CO}_2} = \frac{CE_{\text{daily}}}{G_{\text{daily}}} \quad (3.1)$$

Where CE_{daily} is the daily carbon emission of Ethereum network, and G_{daily} is the daily gas used of Ethereum network.

This equation gives the carbon-per-gas metric so it can be used to calculate CF_{CO_2} , carbon footprint of each transaction with the equation below.

$$CF_{CO_2} = G_{total} \times CE_{unit-CO_2} \quad (3.2)$$

Where CF_{CO_2} is the carbon footprint of a given transaction and G_{total} is the total gas used by the transaction.

Digiconomist API For Ethereum only, this project also uses a second method, 'the carbon footprint per unit of gas in grams of CO_2 ' (Gas_unit_gCO2) factor from the Digiconomist API (Digiconomist n.d.), to double check the efficiency of the proposed model.

$$CF_{gCO_2} = G_{total} \times Gas_unit_gCO2 \quad (3.3)$$

Where CF_{gCO_2} is the carbon footprint in grams of a given transaction and G_{total} is the total gas used by the transaction.

3.7 Summary

This chapter detailed the project's methodological framework, establishing the design goals and the experimental variables necessary to measure the middleware's effectiveness. The primary focus was placed on the simulation framework and the specific mathematical models used for calculating gas consumption and converting it into a tangible carbon footprint. With the analytical foundation now defined, the subsequent chapter will focus on the technical implementation of the on-chain and off-chain layers, detailing the smart contract logic and the simulation workflow used to generate experimental results.

Chapter 4

Implementation

The implementation of the proposed model is structured as three integrated subsystems an on-chain execution layer, an off-chain batching middleware, and a data analysis layer to cleanly separate the on-chain smart contracts, off-chain middleware, and the analytic tools. This separation of concerns ensures that the simulation environments for both networks remain modular.

```
CarbonOptimisedBatcher/  
├── analysis/ ..... Python scripts for data processing and visualization  
├── on-chain/ ..... Hardhat project containing Solidity smart contracts  
│   ├── contracts/ ..... Source code for BatchTransfer and custom tokens  
├── off-chain/ ..... Next.js application acting as the middleware/batcher  
│   ├── app/ ..... Frontend interface for interactive demonstration  
│   └── simulation/ ..... Core simulation logic and batching engines  
└── scripts/ ..... Simulation preparation scripts
```

The on-chain layer is implemented using Hardhat. This layer is responsible for the implementation of token transfer and batching logic. It handles the creation and deployment of the smart contract for both networks.

The off-chain layer is the core layer of this project, it includes the batcher middleware. This layer is implemented as a Next.js based application. It is responsible for executing simulations, communicating with the smart contracts and the networks, and keeping the simulation logs for analysis purposes. The off-chain layer also includes a front-end interface that simulates a twelve-second simulation to demonstrate how batching works.

The final subsystem is the Python based data analysis layer. This layer processes the simulation logs, which were gathered by the off-chain layer, then visualises and summarises the results. It also applies a gas-to-carbon conversion model to reveal the carbon emission reduction of the proposed model.

As illustrated in Figure 4.1, together, these three systems form a modular experimental framework that isolates execution, coordination, and evaluation tasks.

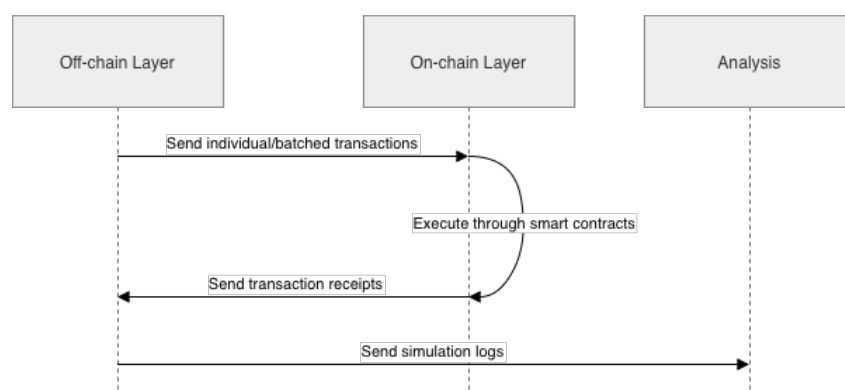


Figure 4.1: Basic visualisation of the interactions between the layers (Created by the author).

4.1 On-chain Layer

This layer acts as the foundational blockchain environment, responsible for processing individual and batched transactions. This layer is built upon Hardhat. Hardhat serves as the primary tool for the on-chain layer. It is a widely used development framework for compiling and executing smart contracts (Nomic Foundation n.d.).

To execute the transactions for Ethereum simulations, the system relies on the Hardhat node. The Hardhat node is a local Ethereum network node designed for development. It runs a local implementation of the Ethereum Virtual Machine, acting as an isolated sandbox, free of charge without any limitations. Mainnet forking is a powerful feature that duplicates the state of the live Ethereum mainnet network onto the local development environment. By integrating an RPC provided by Alchemy, the local Hardhat node fetches the real-time state of the Ethereum mainnet, including smart contracts and tokens already deployed to the network (Alchemy Insights n.d.). This means when forking the mainnet, the Hardhat node has the USDC token smart contract pre-deployed.

Although Hardhat is very useful for compiling and deploying smart contracts on the VeChain network because of VeChain's compatibility with Ethereum, VeChain does not provide mainnet forking and the Hardhat node. To create a sandbox to execute the simulations, VeChain offers the Thor solo node. A Thor solo node offers developers a powerful sandbox environment to interact with the VeChain blockchain. And unlike the Hardhat node, the Thor solo node must be containerised by using Docker (Docker 2018).

However, the key limitation when working with the Thor solo node is that unlike the Hardhat node, the Thor solo node does not fork the VeChain Thor mainnet resulting in the local environment starting entirely empty, without any inherited real-world state or pre-deployed smart contracts except for the hardcoded smart contracts. Consequently, the VIP-180 standard USDC token smart contract has to be manually deployed to the solo node. Since USDC is deployed on the VeChain mainnet, VeChain already provides all the technical aspects of the USDC token (such as source code, ABI, bytecode, etc.) via the VeChainStats block explorer (VeChainStats n.d.). To guarantee that the local simulations replicate the exact smart contract logic of the live network, the original bytecode and ABI were extracted and used to manually deploy the token directly to the solo node before executing any simulations.

To conduct simulations, a standardised set of 150 wallets is utilised across both networks to ensure experimental parallelism. When initiating the Hardhat node for Ethereum, Hardhat automatically creates 150 wallets that are pre-funded with 10,000 ETH. To match this on VeChain, a custom script was developed in the off-chain layer (Subsection 4.2) that programmatically creates 150 wallets for the Thor solo node. The script is run only once, and it saves all the wallets in a file to be used for simulations.

The mechanism for funding these wallets with USDC differs significantly between the two networks due to the different deployment strategies used. On Ethereum, the USDC token's smart contract exists in the Hardhat node since it is forking the mainnet and the smart contract is already deployed there. Because the actual mainnet USDC contract is utilised, the system lacks the administrative authority to mint new tokens. To solve this problem, the simulation transfers 100,000 USDC into each of the 150 wallets created, using an existing mainnet wallet that holds a massive supply of USDC.

On the VeChain network, the USDC smart contract was deployed directly to the Thor solo node manually using its on-chain bytecode that was gathered from VeChainStats (the VeChain block explorer) (VeChainStats 2026). This manual deployment grants the system full control over the token, which means that the system can use any function written in the smart contract, including minting, without any restrictions. As a result, another script was created in the off-chain layer (Subsection 4.2) to fund all 150 wallets by minting 100,000 USDC for each of them. Furthermore, because the wallets have to pay for their own gas fees for individual transactions, the script also transfers 10,000 VTHO (the gas payment token of VeChain) from the solo node's default god wallet (a wallet that has infinite VET and VTHO) to cover their transaction fees.

Smart Contracts

The proposed batching system requires a dedicated on-chain smart contract to receive the batched transactions, unpack, and execute each of them. The USDC token smart contract natively processes only one-to-one transfers and does not support multi-sender token transfers in a single function call, like other ERC-20 and VIP-180 tokens. To resolve this, two custom batch transfer smart contracts were created using Solidity for both networks and they are deployed on both the Hardhat node and the Thor solo node. Although two separate smart contracts had to be written for each network, the code of the smart contracts was designed around the same logic. Both smart contracts essentially receive four arrays: senders' addresses, recipients' addresses, amounts, and signatures. After receiving the batch, the smart contracts go through each of them. For each transfer, smart contract first check if the arrays have the same length since they have one element for every transaction in the batch. (See Appendix A for the full source code of the batcher implementations.)

After the array length verification, the contract processes each transfer. Because these transfers are submitted to the network by the off-chain batching middleware rather than the original senders, the smart contracts must independently authenticate that each sender explicitly authorised their transfer. This is achieved through cryptographic signature verification. A signature is essentially a unique digital fingerprint created by the user to prove they approved the transaction. First, the transaction details — sender, receiver, amount, and nonce — are gathered and converted into a fixed-length code called the message hash. The nonce is a unique, incrementally increasing integer tied to the sender's address. It is included in the message payload and increments after every successful verification, ensuring that each signature is strictly single use. The sender then signs this message hash using their private key for authentication and it gets included in the batch.

Upon receiving the batch, the smart contracts must independently reconstruct these message hashes of the signatures on-chain to verify senders' identities. However, the cryptographic hashing algorithms used to process these signatures differ between the two networks due to their underlying architectural standards. For example, Ethereum relies on the Keccak256 hashing algorithm, which is the native hash function of the EVM. Then, to comply with Ethereum standards and ensure security, the hash is prepended with the EIP-191 standard prefix "`\x19Ethereum Signed Message:\n`" and hashed again. This prefixing mechanism guarantees that the signature was explicitly created for an Ethereum transaction and cannot be executed on a different network (Swende & Johnson 2016).

While VeChain is EVM-compatible, it requires Blake2b256 to be used as the hashing algorithm rather than Ethereum's Keccak256. Furthermore, VeChain enforces VIP-190 for message signing. VIP-190 also has a standard prefix, which is very similar to EIP-191. VIP-190 also applies a prefix; however, it is different from EIP-191 "`\x19VeChain Signed Message:\n`" (Britz 2019).

To verify its authenticity, the smart contracts reconstruct the same message by using the provided transfer details, then they hash the reconstructed message. After hashing it, the contracts must split the provided signature into its cryptographic components (r , s , and v). Using Solidity's inline assembly, the contracts extract these values directly from the transaction data. The variables r and s contain core signature data, while v acts as a 1-byte recovery identifier (Swende & Johnson 2016).

A specific adjustment to this v value is required for the VeChain simulation. The off-chain simulation generates signatures using the VeChain SDK, which natively outputs a raw v value of 0 or 1. However, because VeChain utilises Ethereum Virtual Machine architecture for smart contracts, it relies on Solidity's native `ecrecover` function, which strictly expects a v value of 27 or 28. To resolve this compatibility issue, the off-chain script adds 27 to the v byte of the signature before submitting the batch to the network. This adjustment is strictly a formatting alignment for the EVM and does not alter the cryptographic integrity or validity of the original signature.

Once extracted and properly formatted, these components (r , s , v), along with the generated message hash, are passed into the `ecrecover` function. This function is built into the Solidity programming language and returns the public wallet address of the sender. Finally, the smart contracts check whether the recovered signer wallet address matches with the provided senders' address. If they match, the contract executes the transfer function of the USDC smart contract on behalf of the sender.

4.2 Off-chain Layer

The off-chain layer serves as the central control unit of the project. It serves as both the simulation engine and the batching middleware. Developed with the Next.js framework, this layer interacts with both the local Hardhat node and the Thor solo node to manage the experimental flow (Vercel n.d.).

The off-chain layer contains two execution models: an interactive demonstration for real-time visualisation, and a simulation system for gathering experimental results. The interactive demonstration is integrated into a frontend interface, executing a short, 12-second simulation that executes individual transactions and batches them at the end of 12 seconds to demonstrate the core logic of the proposed system and immediate gas savings. In contrast, the simulation system is not connected to any frontend and it runs in the terminal for extended periods. Simulations execute thousands of transactions across both the Ethereum and VeChain networks to generate statistical data required for evaluation. Detailed implementation of these simulation engines for both networks are provided in Appendix B.

Simulations

Since the execution logic between Ethereum and VeChain varies, separate simulations had to be created for both networks. While both simulations have the same logic, the method of constructing, signing, and submitting the transactions differs to comply with the underlying network protocols. While the Ethereum simulations use Ethers.js for all network interactions and cryptographic operations, the VeChain simulation employs Ethers.js as the common utility and relies on the VeChain SDK to execute core blockchain functions and transaction management (*Ethers.js Documentation* n.d., VeChain Foundation 2025a).

Despite the architectural differences between the two networks, the core logic of the simulations remains identical. This ensures that any observed variance in gas consumption is strictly a result of network differences. To maintain this strict parity, both simulation engines are initialised using a set of configuration parameters sourced from the central environment (.env) file. This approach allows rapid reconfiguration between simulation runs without code modification. The simulations require the batch size (n) and batch interval (t) to control the batching system; the simulation duration, to define the scope of data collection; and the throughput (λ), which dictates the rate of transaction generation in transactions per second (TPS).

Batching

Submitting transactions to the VeChain network differs significantly from Ethereum due to its unique multi-task transaction model. In VeChain, transactions are structured using clauses. A single transaction can contain multiple clauses, where each clause represents a distinct action, such as a token transfer or smart contract interaction. This allows a single user to execute multiple tasks with one transaction (VeChain Foundation 2025b).

However, this native clause mechanism cannot be used to batch transactions for this project. This limitation is because a VeChain transaction is signed by a single private key, meaning all clauses within that transaction must originate from the single wallet that signs the transaction. This means, clauses allow one sender to send tokens to multiple different wallets, whereas they do not allow multiple different senders to be bundled into one. Since the primary objective of this project is to batch transactions from 150 different wallets, relying solely on native clauses is not sufficient. Therefore, collecting the transfers into one clause and sending them to the smart contract to execute all of them remains necessary.

In the Ethereum simulation, the batching process relies on EIP-191 message signing. The helper function responsible for executing batches iterates through the batch, constructing a unique message hash for verification. The message hash is generated using Keccak from Ethers.js, which combines the sender, recipient, amount, and nonce. Keccak is a family of cryptographic hash functions based on the sponge construction. It is implemented using a permutation-based design derived from a set of seven permutations (EL Makhtout & Bentaleb 2022). After gathering the message hash, this hash is signed by the sender's wallet according to the ERC-191 standard. Once all the signatures are collected, the

sender, recipient, amount, and signatures are put into parallel arrays. Then these values are passed to the batch executor smart contract by the batcher wallet for the transfers to be submitted to the network in a single transaction.

Conversely, the VeChain simulation implementation requires a hybrid approach to bridge EVM compatibility with the network's requirements. While Ethers.js is still used for organising transaction arguments into bytes, the VeChain simulation utilises Blake2b hashing via VeChain SDK. Blake2 is an ARX-based cryptographic hash function and Blake2b is one of the two main algorithms of Blake2, designed for 64-bit architectures and capable of generating hash outputs up to 64 bytes length (EL Makhtout & Bentaleb 2022). After hashing the arguments, each sender signs the message according to the VIP-190 standard. Once all the signatures are collected, the sender, recipient, amount, and signatures are stored in parallel arrays, similar to the Ethereum simulation. Finally, the function call is wrapped in a Clause and put in a transaction body for the god wallet to sign and send the transaction to the network. The god wallet is the admin wallet of the VeChain Thor solo node that serves as the gas payer for batches because it holds unlimited VET and VTHO.

Both simulations also employ nonce tracking locally to ensure that a sender can get included in the same batch multiple times. A local nonce map keeps track of every sender's nonce by initially fetching the current nonces from the smart contract. If the same sender appears in the batch twice, the system increments the cached nonce locally for each subsequent transaction. This prevents nonce collisions and ensures that every transaction from the same user is executed successfully.

Simulation Workflow

The core workflow follows a while loop structure that continues until the current time exceeds the defined simulation end time. Within each iteration, the system first checks if the number of collected transactions in the current batch has reached the batch size N or if the time elapsed exceeded the batch interval t . If the batch size is reached or the batch interval is exceeded, the system automatically executes the batch.

After completing the condition check and executing any pending batch if required, the simulation proceeds to generate a new transaction using a dedicated helper function. The helper function constructs a randomised transaction object by selecting a sender and receiver from the pre-generated pool of 150 wallets and generating a random transfer value between 1 and 3 USDC. The transaction generation logic remains identical for both networks to ensure experimental consistency. However, the execution of the individual transactions differs to comply with the specific architectural requirements of each network. In the Ethereum simulation, the system immediately executes the standard ERC-20 transfer function, submitting the transaction to the local Hardhat node. To capture the baseline energy cost, the system awaits the confirmation of the transaction receipt to extract the gasUsed value before logging the transaction data.

In contrast to the Ethereum simulation, the execution logic for the VeChain simulation is adjusted to accommodate the network's deterministic block generation rule. Under VeChain's PoA model, block generation cannot be performed by anonymous nodes. Only Authority Masternodes are allowed to generate blocks. This adheres to a strict schedule, where a new block is generated every 10 seconds. For each interval, VeChain implements a Deterministic Pseudo-Random Process (DPRP) to select a specific Authority Masternode to propose the block, ensuring that all active validators have an equal opportunity to participate in the consensus mechanism (VeChain Foundation 2024). Consequently, individual transactions submitted to the Thor solo node are not executed immediately. Instead, they enter the memory pool (mempool) where they remain pending until the next block is proposed. To accommodate this, the simulation submits the transactions asynchronously without awaiting immediate confirmation. The simulation pauses to wait for the conclusion of the 10-second block interval. Once the block is committed, the simulation retrieves the transaction receipts in bulk to record the gas consumption data for all transactions executed within that block.

If the individual transaction executes successfully, its data is temporarily stored in an off-chain buffer, and the transaction is added to the pending batch array. Conversely, if the transaction fails during execution, the error is caught, the transaction is not added to the buffer or to the batch array, and the loop

proceeds. Crucially, the baseline data held in the temporary buffer is not committed to the simulation log until the aggregated batch is subsequently executed by the smart contract without any issues. This buffer mechanism ensures that if a batch fails, its corresponding individual transactions are not recorded, preventing invalid data from skewing the final statistical analysis.

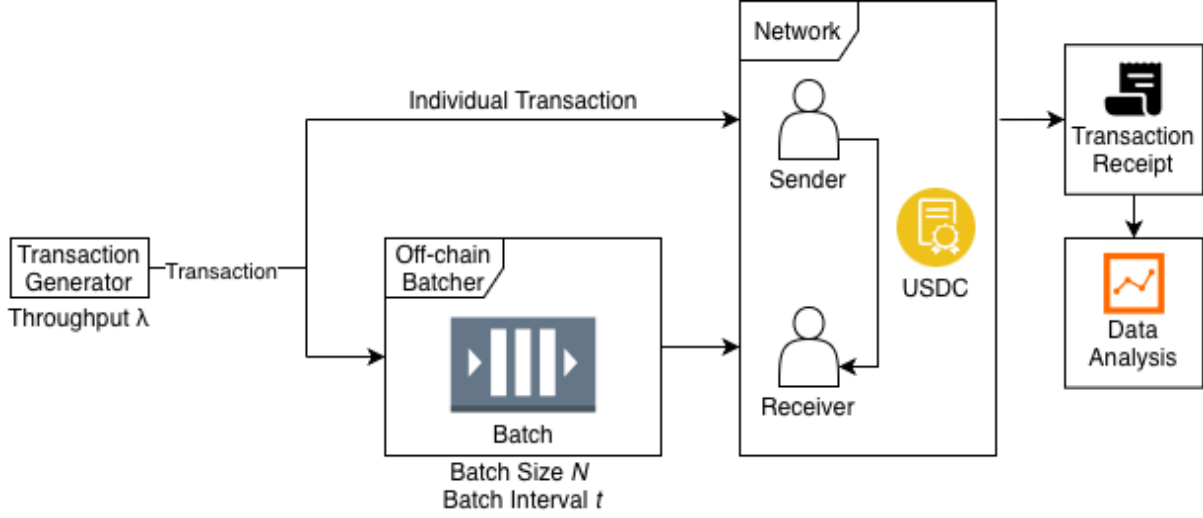


Figure 4.2: Basic visualisation of simulation workflow (Created by the author).

Following the execution and validation of the individual transaction, the simulation enforces a calculated time delay before the loop iterates again. This delay is implemented to control the simulated network throughput (λ). Instead of using a fixed wait time, the system generates randomised intervals between each iteration to accurately model the unpredictable arrival times of real-world network traffic. To ensure these random delays average out to correctly match the target simulation throughput, the system models the transaction arrivals as a Poisson process.

According to Law et al. (2007), in a Poisson process with an arrival rate of λ , the time intervals between successive events follow an exponential distribution with a mean of $1/\lambda$. To computationally calculate these specific time delays, the system employs the inverse-transform method. Law et al. (2007) demonstrates that an exponential inter-arrival delay can be generated from a uniformly distributed random number, $U \sim (0, 1)$, using Equation 4.1.

$$delay_{ms} = \frac{-\ln(U)}{\lambda_{ms}} \quad (4.1)$$

Where U is a uniformly distributed pseudo-random floating-point number in the range $[0,1)$, and λ_{ms} is the target transaction throughput converted to a millisecond rate. By calculating the delay, the simulations accurately mimic organic network congestion while maintaining the correct target throughput. To enforce this delay at the conclusion of each transaction cycle, the simulations employ the TypeScript function detailed in Listing 4.1.

```

1 function getPoissonDelay(targetTPS: number): number {
2   // Prevent division by zero fallback
3   if (targetTPS <= 0) return 3000;
4
5   // Convert throughput rate to milliseconds
6   const ratePerMs = targetTPS / 1000;
7
8   // Inverse-Transform Sampling formula: -ln(U) / λ
9   const delayMs = -Math.log(Math.random()) / ratePerMs;
10
11   return delayMs;
12 }

```

Listing 4.1: TypeScript implementation of Inverse-Transform Sampling for Poisson delay.

Simulation Preparation

Before running the simulations to test the proposed middleware system, the networks and simulation environments should be prepared. Because the underlying node architectures differ, the preparation process requires a different sequence of operations for each network. To handle these operations, two TypeScript scripts are added to the project at the root level. Placing simulation preparation scripts at the root level is a deliberate architectural choice. Because these scripts must bridge multiple environments, they function as independent coordination layers that govern the entire project setup.

The first requirement is to deploy the required smart contracts to the local networks so they possess an active on-chain address for the off-chain system to interact with. Both scripts deploy the custom batcher smart contract. However, since the VeChain Thor solo network is not forking the mainnet like the Hardhat node does, for the VeChain simulation the USDC token's smart contract has to be deployed before the batcher smart contract using its on-chain bytecode.

After deploying the smart contracts, the scripts proceed to fund all the wallets with 100,000 USDC. On Ethereum, this process is done by a whale wallet that contains a large amount of USDC, while on VeChain, since the USDC smart contract was deployed by the god wallet, it has the authority to mint as much USDC as needed directly to the wallets. The script for VeChain also transfers 10,000 VTHO to each wallet. This is a mandatory step because when the framework simulates individual transactions, every wallet has to pay for its own network gas fees to process the transfer. The reason the Ethereum preparation script does not transfer ETH to wallets is that the Hardhat node already funds every wallet with 1000 ETH on initialisation.

The final step of preparation is granting the batching smart contracts the cryptographic permission to move the users' funds. The proposed system relies on a central smart contract to receive a batch, unpack the arrays, and execute transfers on behalf of the original senders. However, under both ERC-20 and VIP-180 token standards, a third-party smart contract cannot arbitrarily withdraw tokens from a user's wallet. To resolve this, the preparation scripts iterate through all 150 simulation wallets and submit an `approve` transaction for each one. This grants the newly deployed batcher smart contract a specified allowance, explicitly authorising it to utilise the token's `transferFrom` function to move USDC on the user's behalf during simulations.

Once these automated tasks are executed with no errors, both local nodes are populated with the necessary smart contracts, funded wallets, and permissions, allowing the simulations described in Section 4.2 to run securely.

4.3 Summary

Chapter 4 presented the technical development of the project by dividing the system into three main layers: on-chain execution, off-chain middleware, and data analysis. The implementation included creating Solidity smart contracts for batch transfers and setting up cryptographic authentication to support both Ethereum and VeChain. It also defined the simulation process, using Inverse-Transform Sampling to represent realistic network traffic patterns. With the system implemented and the preparation scripts completed, the next chapter will analyse the simulation results to evaluate the carbon emission reductions achieved.

Chapter 5

Results and Evaluation

The evaluation of the proposed off-chain batching middleware is based on the data generated by the simulations across both the Ethereum and VeChain networks. To systematically assess the system's carbon emission, the simulations are executed under controlled configurations, defining specific values for batch size, batch interval, and network throughput. The transaction data collected are then analysed to measure the computational gas consumption of individual transfers compared to batched transfers. This raw gas data is subsequently processed using the gas-to-carbon conversion method established in Subsection 3.6 to quantify the reduction in carbon emission. Finally, the resulting data is used to prove the carbon efficiency of the proposed system and find the optimal batching configuration to achieve the highest carbon emission reduction.

```
1 {
2   "simulationStartTime": DateTime,
3   "simulationEndTime": DateTime,
4   "simulationDuration": DateTime,
5   "batchSize": integer,
6   "batchIntervalMinutes": DateTime,
7   "throughput": integer,
8   "individualTransactions": [
9     {
10      "sender": string,
11      "recipient": string,
12      "amount": string,
13      "gasUsed": string,
14      "timestamp": DateTime
15    }
16  ],
17  "batches": [
18    {
19      "batchNumber": integer,
20      "gasUsed": string,
21      "timestamp": DateTime,
22      "transactionCount": integer,
23      "transactions": [
24        {
25          "sender": string,
26          "recipient": string,
27          "amount": string,
28          "timeStamp": DateTime
29        }
30      ]
31    }
32  ],
33  "summary": {
34    "totalIndividualTransactions": integer,
35    "totalBatches": integer,
36    "totalIndividualGasUsed": string,
```

```

37     "totalBatchGasUsed": string
38   }
39 }

```

Listing 5.1: JSON schema structure of the output simulation logs.

To facilitate accurate analysis, the off-chain simulation engine is designed to continuously monitor transaction executions and compile the raw data into a structured JSON log at the end of each simulation run. This comprehensive logging ensures that all variables and outcomes are preserved symmetrically for both the individual and batched execution paths. The generated simulation logs are systematically organised into four primary components: simulation parameters, individual transaction logs, data for every batch, and aggregated summary of the simulation.

As detailed in Listing 5.1, the root of the JSON structure captures the core independent variables governing the specific simulation run. These simulation parameters include the start and end timestamps, the total duration of the simulation, and the exact configuration tested in that instance: batch size N , batch interval t , and throughput λ . Recording these parameters is used to tie every JSON file to its unique experimental conditions.

Following the configuration data, the simulations systematically log the transaction level metrics for both individual transactions and batches to isolate the outputs for each execution model. For the individual transactions, the system records the sender and the recipient wallet addresses, the transfer amount, the execution timestamp, and the gas used for that transaction. Concurrently, for the proposed middleware path, the batch data array records the gas used for each executed batch commit, the total number of transfers, the execution timestamp, and a nested array detailing the original token transfer, ensuring data traceability for batched transfers.

Finally, the simulation log concludes with an aggregated summary that provides a statistical overview of the entire simulation. The summary shows the total number of individual transactions processed and the total number of batches executed. It aggregates the total gas consumption for both the individual execution path and the proposed middleware path. This final summary forms the direct mathematical foundation for the subsequent analysis of gas efficiency, allowing for a highly controlled comparison of carbon consumptions.

5.1 Experimental Overview

To thoroughly evaluate the viability and environmental impact of the proposed middleware, numerous simulations were executed across both the Ethereum and VeChain networks. To ensure a fair comparison, identical simulation setups were applied to both networks.

A critical parameter manipulated throughout the experiments is the batch size N . After conducting a series of stress tests, the maximum viable batch size was determined to be 150 on the Ethereum network and 200 for the VeChain network. Attempting to aggregate more transfers into a single batch resulted in payload sizes that exceeded the computational processing thresholds of the respective local nodes, leading to batch execution failures. Also, based on their research, Wang et al. (2021) claims that to achieve a reduction in gas cost, the batch size should be minimum 5. Therefore, the simulations in this study begin at a batch size of five and are incrementally scaled up to 150 to observe the correlation between batch size and carbon emission reduction.

To assess the robustness of the middleware under different network conditions, the simulations test three distinct throughput rates (λ): 2, 5, and 9 transactions per second. These rates were systematically generated using the Poisson distribution algorithm to simulate low, average, and high network congestion states.

Alongside these variables, the batch interval t was held constant at 5 minutes across all experimental runs. Functioning as a safety net, the batch interval prevents infinite waiting times for transactions inside the batches. It ensures that during low network congestion, transfers would not be delayed indefinitely if the batch size is not reached.

5.2 Data Analysis

To systematically evaluate the performance of the middleware system, a Python-based data analysis pipeline was developed. This pipeline is responsible for parsing the JSON simulation logs generated by the off-chain layer, extracting core metrics, and performing the mathematical conversions required to quantify the system's carbon emission reduction.

The analysis begins with data ingestion and initial preprocessing. The pipeline scans the designated log directories for each network and verifies the structural integrity of the JSON files. It ensures that batch executions are recorded sequentially without interruptions, and any simulation logs exhibiting non-sequential batch numbers are discarded to prevent skewed statistical outcomes. Once validated, the data is passed through gas analysis and latency analysis modules.

1. **Gas Analysis:** This module evaluates the computational efficiency of the system by extracting the pre-aggregated totals directly from the simulation logs. It compares the total gas of individual transactions against the total gas consumed by the middleware's batched execution.
2. **Latency Analysis:** Because latency is a designated trade-off in this proposed architecture, it is strictly monitored. The module calculates the exact waiting time for every individual transaction by subtracting the initial transaction timestamp from the final batch execution timestamp. This granular data is then aggregated to find the average, minimum, and maximum latencies across different batch sizes and throughput rates.

Carbon Emission Conversion

The fundamental objective of the analysis is translating these gas savings into tangible carbon emission reductions. For the Ethereum network, the pipeline employs a dual-verification methodology described in Subsection 3.6. The primary conversion method dynamically calculates this ratio using Ethereum's daily estimated carbon emission provided by Cambridge Centre for Alternative Finance (n.d.) and daily network gas usage provided by etherscan.io (2015). These data are extracted by the pipeline using the CSV files, and to achieve an accurate result, the pipeline uses the most recent date that exists on both of the datasets. To validate the primary conversion method, the script executes a secondary method by querying the Digiconomist API (Digiconomist n.d.) to fetch its independently tracked carbon-per-gas metric for the exact date used for the primary conversion method. The pipeline calculates the final CO_2 reduction using both models side by side, allowing the visualisations to plot the models against each other for cross examination. On the other hand, the VeChain analysis employs the primary conversion method only. For the VeChain analysis, daily carbon emission and gas usage are fetched through the API provided by VeChainStats (n.d.).

5.3 Ethereum Analysis

The Ethereum network simulation results demonstrate a correlation between off-chain batching and a reduction in both computational gas and carbon emissions. Based on Figure 5.4, regardless of the network congestion simulated, increasing the batch size systematically decreased the relative carbon footprint per simulation. Furthermore, the data indicates that the carbon reduction percentages are independent of transaction throughput. Whether the network operates at 2, 5, or 9 TPS, the proportion of the carbon savings remains consistent across all throughput levels.

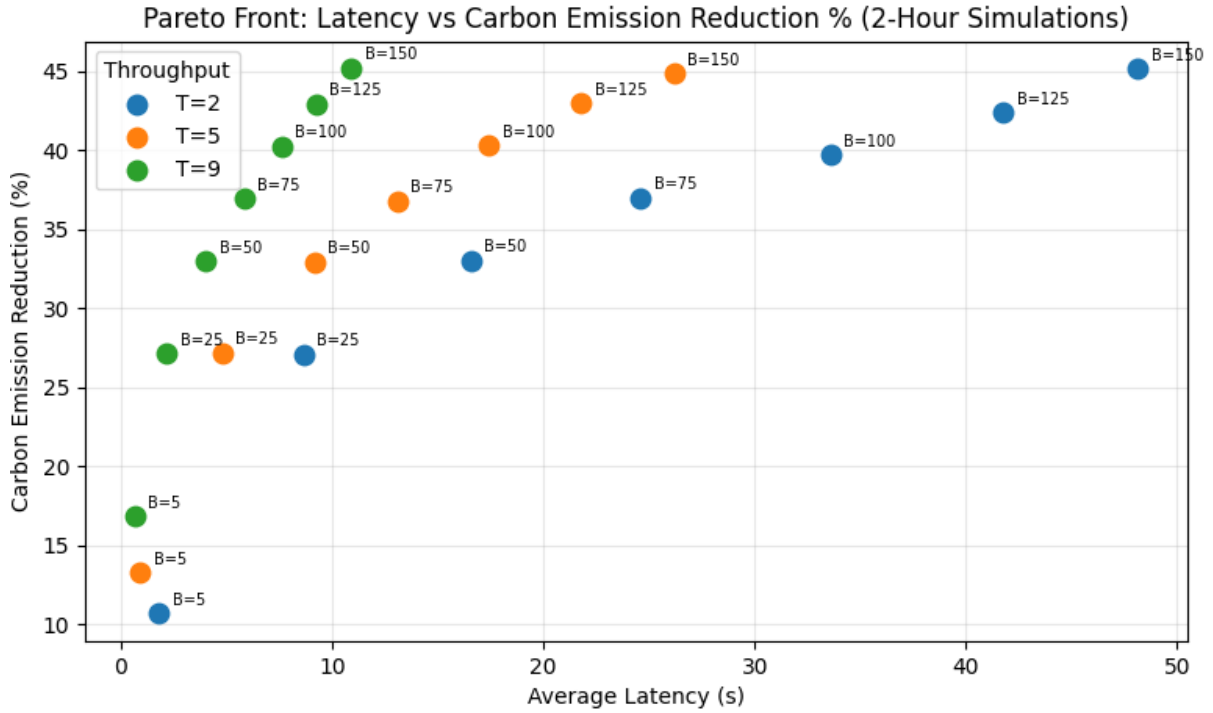
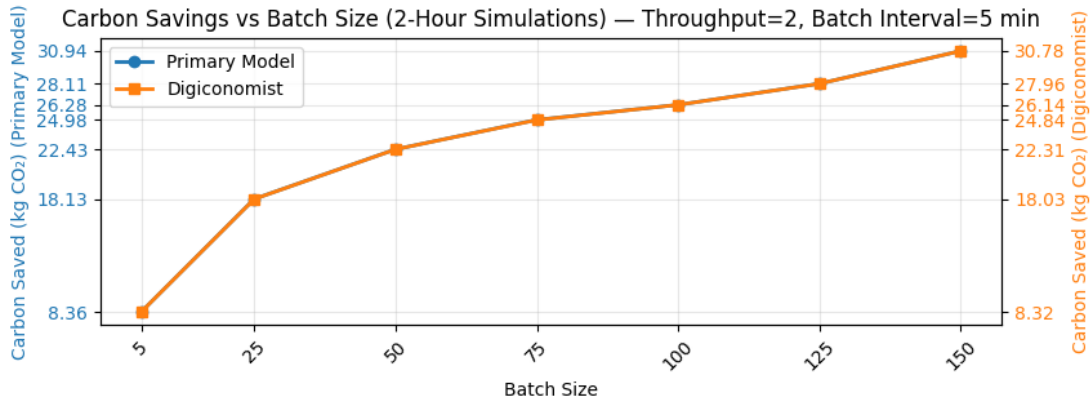


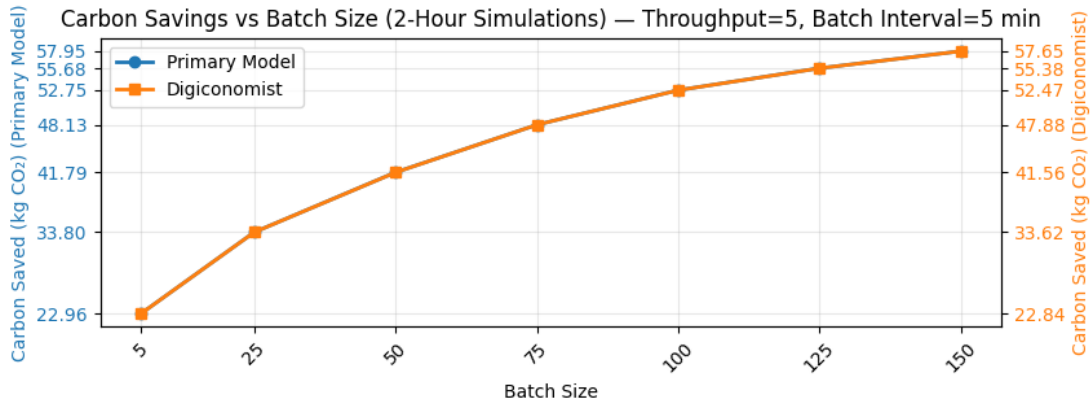
Figure 5.1: A Pareto front scatter plot illustrating the trade-off between average latency and carbon emission reduction percentages across varying throughput (T) and batch size (n) configurations for the Ethereum network.

However, a notable anomaly was observed at the lowest evaluated batch size of five. While the system achieved a baseline carbon emission reduction percentage of 10.7%, this percentage is not uniform across all throughputs. Because the system continuously processed incoming transactions every second, higher throughput rates (5 and 9 TPS) caused a significant number of batches to overflow the intended batch size of five before the batch execution could finalise. As observed from the simulation logs, at 2 TPS, there were few to no overflowed batches. At 5 TPS, a moderate amount of overflowed batches occurred, and at 9 TPS, the rapid influx of transactions resulted in a substantial number of overflowed batches. Because these overflowed batches effectively processed more than 5 transactions in practice, the observed carbon reduction percentage increased alongside the throughput. This ultimately demonstrates that if the network throughput is equal to or exceeds the batch size, transactions accumulate faster than the execution of the batch finalises, causing the batches to overflow and rendering the system unstable.

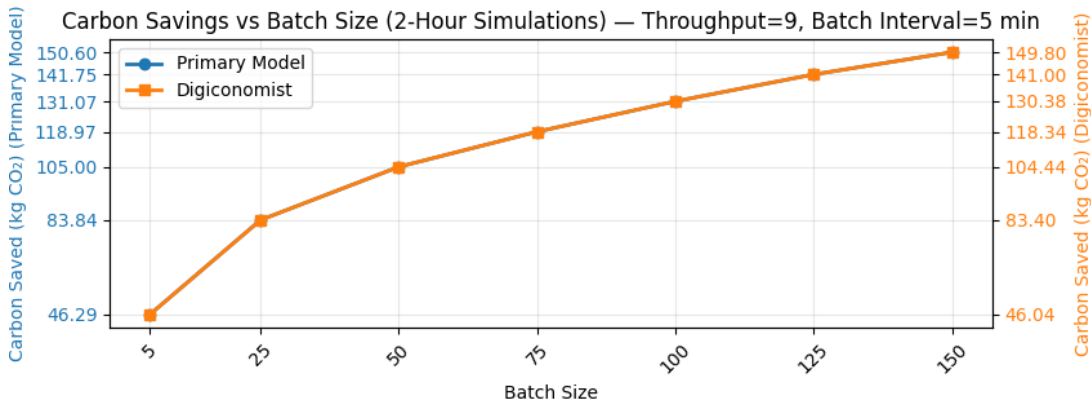
The maximum carbon reduction was consistently achieved at a batch size of 150. This aligns with the system's maximum viable threshold on the Ethereum network before encountering payload execution failures. At this optimal batch size, the middleware recorded a peak carbon emission reduction of approximately 45% across all throughputs.



(a) Throughput = 2



(b) Throughput = 5



(c) Throughput = 9

Figure 5.2: Line graphs illustrating the relationship between batch size and carbon savings (kg CO_2) for both the primary and Digiconomist models across two-hour simulations at throughput levels of 2, 5, and 9 for the Ethereum network.

To validate the accuracy of the estimated carbon reductions, the primary gas-to-carbon conversion method was plotted alongside the secondary Digiconomist API method. As seen in Figure 5.2, while not identical, the results from both conversion methods aligned almost perfectly across all tested throughput levels. The variance between the calculated values was found to be statistically insignificant ($p < 0.05$). This tight correlation confirms the mathematical validity of the primary conversion method, establishing a reliable baseline for the performance analysis for both the Ethereum and VeChain networks, since the primary conversion model is the same for VeChain as Ethereum.

Ethereum Latency Analysis

While the proposed off-chain batching middleware successfully reduces the carbon emission, it inherently comes at the cost of increased transaction latency. Because individual transactions are held in the batch until the target batch size is reached or the batch interval expires, the user experiences a mandatory delay before their transaction is committed to the layer-one network.

As illustrated in Figure 5.4, transaction latency scales linearly with batch size. Across all three throughput scenarios, scaling the batch size from 5 to 150 yields a strict increase in average latency. This proportional increase is a fundamental characteristic of the proposed system's batching mechanism. A larger batch size requires the off-chain middleware to hold the batch for submission longer to accumulate the necessary payload. Consequently, the first transactions entering a newly created batch experience long delays while waiting for the remaining slots of the batch to fill, whereas the final transaction that triggers the batch execution experiences near-zero latency. This large difference in waiting times between the first and the last transactions explains why Figure 5.4 shows a much wider spread of latencies as the batch size increases.

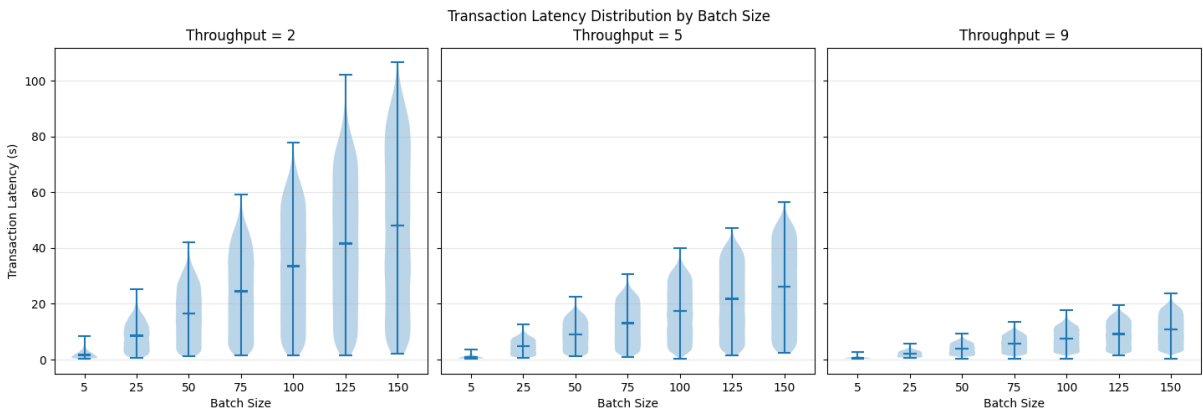


Figure 5.3: Violin plot that demonstrates the range of transaction latencies for throughput levels of 2, 5, and 9 TPS for the Ethereum network.

As Figure 5.3 shows, transaction latency is heavily mitigated by network throughput. Under low congestion (2 TPS), filling the batch size of 150 results in an average latency of approximately 48 seconds. Furthermore, the variance in wait times under low throughput is exceptionally wide, leading to unpredictable user experiences. Conversely, under high network congestion (9 TPS), the middleware accumulates transactions rapidly. At this high throughput, the same batch size of 150 is achieved with an average of only 10.9 seconds, accompanied by a tightly clustered distribution of wait times. This demonstrates that the proposed system performs better during high network congestion.

5.4 VeChain Analysis

Contrary to the positive impact on carbon reduction of the proposed system on the Ethereum network, the VeChain simulations display inverse results. The data demonstrates that the proposed off-chain batching middleware failed to reduce the carbon footprint of transactions on the VeChain network.

When breaking down the carbon savings percentages, the system recorded negative carbon savings across the board. The carbon emission increase is around 20%. The data reveals that there is no continuous correlation between batch size and carbon emission on VeChain. Instead, the proposed system exhibits an initial decrease of carbon emission increase by roughly -26% to -22% from batch size 5 to batch size 25 across all throughputs. As the batch size scales from 25 up to the maximum threshold of 200, the negative reduction percentage flattens and stabilises near the -20% to -21% mark.

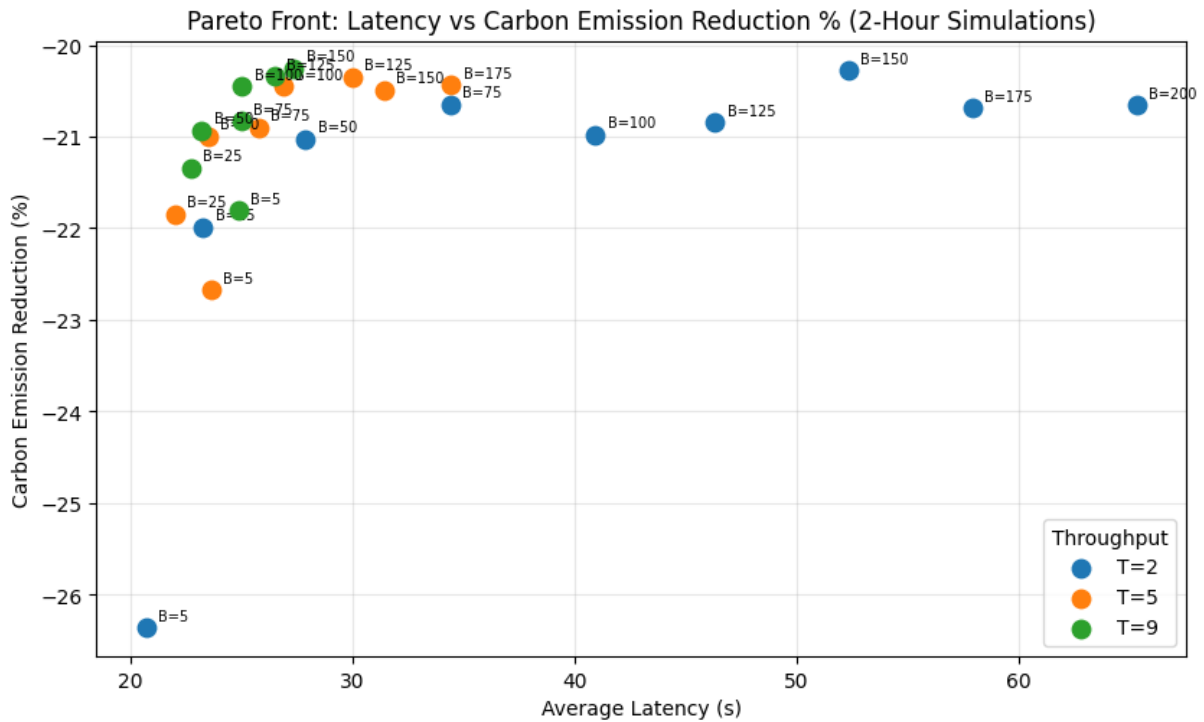


Figure 5.4: A Pareto front scatter plot illustrating the trade-off between average latency and carbon emission reduction percentages across varying throughput (T) and batch size (n) configurations for the VeChain network.

VeChain Latency Analysis

Despite the inverse results in carbon reduction, the transaction latency on the VeChain network largely followed the structural behaviour observed on Ethereum. Because the fundamental logic of the off-chain batching middleware remained identical, transaction latency generally scaled linearly with batch size once the system was stabilised.

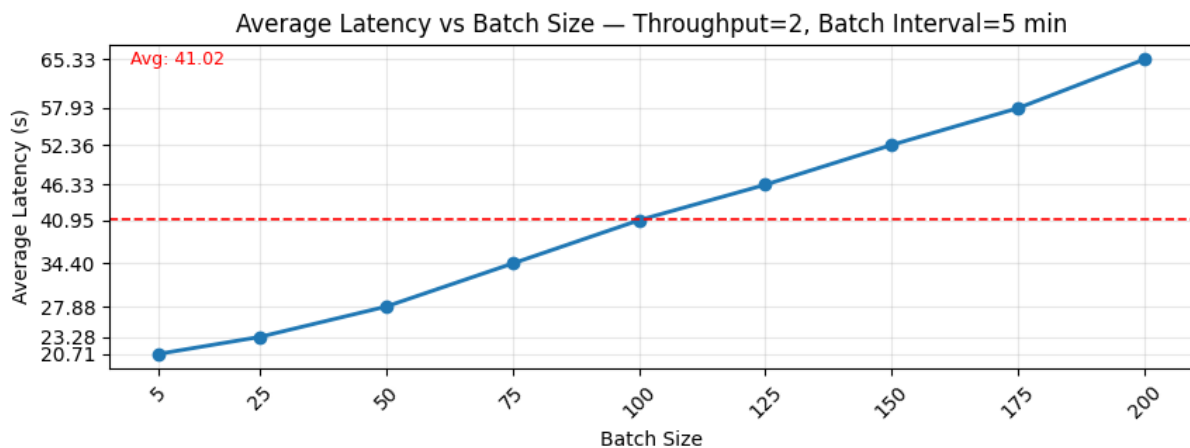
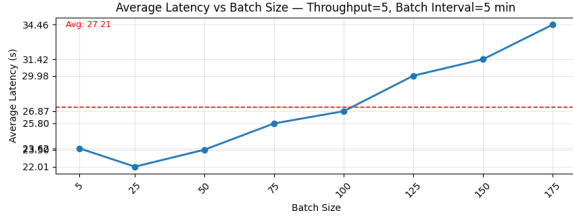
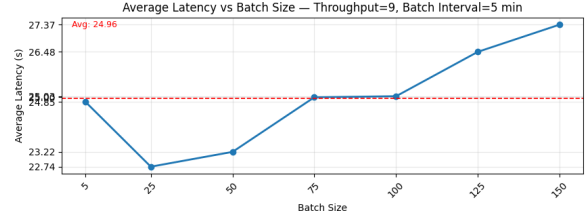


Figure 5.5: A line graph to visualise the relation between latency and batch size.

As shown in Figure 5.5, under low throughput conditions (2 TPS), average latency scaled linearly, starting at roughly 20.7 seconds for the batch size of 5 and peaking at 65.33 seconds for the batch size of 200.



(a) Throughput = 5



(b) Throughput = 9

Figure 5.6: Line graphs illustrating the relationship between batch size and latency across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

However, Figure 5.6 reveals a significant latency spike at the lowest batch size ($N = 5$) under moderate and high throughput conditions (TPS 5 and 9). At 5 TPS, the average latency started high at roughly 23.6 seconds and dropped to 22.0 seconds at the batch size of 25. Then it resumed its expected linear climb. This anomaly was even more visible at 9 TPS, where latency started at 24.8 seconds for the batch size of 5 before dropping to 22.7 seconds at the batch size of 25.

This initial spike indicates system instability of the proposed system on the VeChain network. Because VeChain utilises a strict 10-second block generation rule, submitting batches of 5 when the transactions arrive at 5 and 9 per second completely overwhelms the off-chain middleware. The system attempts to submit batches faster than the network’s mempool and block intervals can finalise them, creating a bottleneck that artificially inflates the wait time. Once the batch size is increased to 25, the middleware accumulates transactions efficiently enough to align with the network’s block schedule, allowing the system to stabilise.

5.5 Conclusion

The simulation results revealed contrasting outcomes for the proposed batching middleware across the two evaluated networks. On the Ethereum network, the system successfully correlated off-chain batching with carbon reduction, achieving a peak carbon reduction of approximately 45% at the maximum viable batch size of 150. These carbon savings remained proportionally consistent across all tested throughput levels of 2, 5, and 9 transactions per second. However, the simulation results displayed an anomaly at the batch size of 5 where high throughput caused batches to overflow, which resulted in system instability. Although the system still decreased the carbon emission, the carbon emission reduction percentages increased at the batch size 5 as the throughput increased. Transaction latency on Ethereum scaled linearly with batch size, though higher network congestion significantly mitigated the latency, reducing the average latency for a batch of 150 from approximately 48 seconds at 2 TPS to 10.9 seconds at 9 TPS.

Conversely, the middleware failed to reduce the carbon footprint on the VeChain network, consistently recording a carbon emission increase of approximately 20% across all configurations. The VeChain data did not show a continuous correlation between batch size and carbon emission reduction, with negative reductions stabilising between -20% and -21% for batch sizes from 25 up to 200. The proposed system demonstrated an anomaly on the VeChain network similar to the Ethereum network, where batch size of 5 caused batches to overflow, which caused even worse carbon emission reduction. Furthermore, while VeChain transaction latency scaled linearly like on Ethereum, moderate and high throughput scenarios (5 and 9 TPS) produced initial latency spikes and system instability at the smallest batch size of 5 before the system could stabilise at the batch size of 25.

In summary, this chapter presented a comparative analysis of gas and carbon data, providing the empirical basis for assessing the system’s effectiveness. These findings lead into the final chapter, which will discuss the architectural drivers behind these results, address the project’s limitations, and outline future research directions.

Chapter 6

Discussion

Before discussing the broader implications of the experimental outcomes, it is essential to review the progression of this research. The technical foundation of the study was established by examining blockchain architectures, the execution of smart contracts, and the mechanics of gas consumption across both the Ethereum and VeChain networks. By identifying the limitations of current transaction models alongside the specific role of stablecoins like USDC, the rationale for a carbon-optimised middleware was solidified. Building upon this, the methodological framework defined the design goals and experimental variables, placing key focus on the simulation framework and the mathematical models required to convert gas consumption into a tangible carbon footprint. The system was then implemented across three main layers: on-chain execution, off-chain middleware, and data analysis. Crucially, this implementation established the simulation workflow, utilizing a dual-path execution model to process and compare individual versus batched transactions simultaneously. This workflow relies on Inverse-Transform Sampling to generate realistic Poisson-distributed network traffic, while fundamentally adapting its execution logic to accommodate Ethereum's immediate processing alongside VeChain's strict 10-second block generation schedule. Finally, a comparative analysis of the resulting gas and carbon data provided the empirical basis for assessing the system's effectiveness. With this foundation established, this chapter will discuss the architectural drivers behind these results, address the limitations of the project, and outline directions for future research.

The fundamental objective of this study was to evaluate whether an off-chain transaction batching middleware system could improve the environmental sustainability of blockchain networks. The experimental outcomes reveal that the efficiency of the proposed system is not universal, but rather dependent on the underlying layer-one network architecture.

On the Ethereum network, the proposed middleware successfully demonstrated a direct correlation between transaction batching and carbon reduction. Through rigorous parameter manipulation, the definitive optimal configuration for maximising the carbon emission reduction was identified as the batch size of 150. At this exact threshold, the system achieved a peak carbon emission reduction of approximately 45%. To contextualise the real-world impact of these parameters, during a high-throughput simulation processing 56,019 individual transactions, this optimal batching configuration successfully reduced 150 kg of CO_2 emission.

This reduction threshold was a result of a constraint dictated by the Ethereum Virtual Machine's block gas limit. As the batch size exceeded 150, the cumulative computational cost of iterating through the payload and executing `ecrecover` for each signature caused the batch transaction to exceed the block gas limit enforced by the network node, resulting in execution failures. The outcome is largely driven by the fixed computational pricing model of the Ethereum Virtual Machine (EVM). Every standard transaction on the Ethereum network incurs a static base cost of 21,000 gas units, regardless of its complexity. By batching multiple transfers into a single payload, the off-chain batcher successfully reduced this heavy, fixed base cost across numerous senders, which mathematically reduces the average gas consumption per transaction. Although the batching smart contract introduces additional execution overhead, the total gas consumed remains significantly lower than executing each transfer individually,

resulting in a net reduction in overall energy consumption.

Conversely, the exact same off-chain batching logic produced the opposite carbon emission reduction results on the VeChain network, consistently increasing the carbon emission by approximately 20%. This highlights how differences in network architecture influence the outcome of the proposed system. VeChain natively supports multi-task transactions via its clause architecture, allowing multiple operations to be bundled under a single base cost of only 5,000 gas units. Because the proposed off-chain batching middleware was designed to collect transactions from multiple senders, it could not rely on VeChain's native single-signature clause mechanism. Instead, it required a custom smart contract to iterate through arrays and independently authenticate each sender using complex cryptographic signature verification, including Blake2b hashing and `ecrecover` executions on-chain.

The analytical data strongly suggest that the computational overhead required to process these cryptographic verifications on-chain exceeded the natively low base cost of standard VeChain transactions. Ultimately, attempting to force an EVM optimised batching workaround onto a network that already possesses a highly efficient native multi-task architecture proved counterproductive to environmental sustainability.

While the system provided high efficiency on the Ethereum network at larger batches, the data revealed a crucial vulnerability when operating at the minimal batch size of 5 under moderate to high network congestion. At throughput rates of 5 and 9 transactions per second, both networks exhibited significant system instability and latency anomalies.

This instability results from a fundamental bottleneck between the rapid arrival rate of incoming transactions and the slower execution speed of the layer-one networks. On both networks, when transactions arrive at a high throughput but the target batch size is smaller than throughput, the middleware accumulates and attempts to submit batches faster than the networks can confirm them. On the Ethereum network, this rapid influx means transactions pile up in the off-chain middleware faster than the EVM can execute and finalise the previous batch, causing pending batch arrays to overflow. This asynchronous misalignment caused the batches to overflow, effectively forcing the system to process more transactions per batch than the configured limit in order to clear the rapidly growing backlog. Consequently, as throughput increases, the degree of batch overflow also rises, resulting in larger effective batch sizes and greater carbon savings.

On the VeChain network, this bottleneck is further dictated by its strict Proof of Authority (PoA) schedule, which emits a new block exactly every 10 seconds. The middleware floods the network's mempool with multiple small batches that simply cannot be cleared until the next block interval. Because batches are filled faster than they can be executed, many are already queued before the current block is finalised. As a result, once the 10-second interval elapses, only one batch can be included, while the rest are forced to wait for subsequent block intervals. This creates a compounding delay, where some of the batches experience multiple 10-second waiting cycles before execution.

This anomaly demonstrates a critical operational limitation. For an off-chain batching system to remain stable, the target batch size must scale proportionally with network throughput. When the influx of transactions exceeds the time required for the blockchain network to confirm the batch, the middleware becomes overwhelmed. This leads to unpredictable latency spikes and distorted carbon reduction metrics.

Latency Trade-off

A core principle of this project was the acceptance of increased latency to prioritise carbon footprint minimisation. The latency analysis on the Ethereum network confirms that this trade-off behaves linearly; as the batch size increases to maximise carbon savings, the average wait time for users increases proportionally.

At a low throughput of 2 TPS, achieving the optimal 45% carbon emission reduction required senders to wait an average of 48 seconds for transaction finality. While this delay is inefficient for real-time financial trading, it represents a highly viable solution for non-urgent tasks, such as payroll distributions, scheduled supply chain data logging, or automated payments. The data further illustrates

that this latency trade-off is heavily mitigated by network congestion. Under a high throughput of 9 TPS, the system achieved the exact same 45% carbon emission reduction with an average delay of only 10.9 seconds. This suggests that the proposed middleware becomes more viable from a user-experience perspective as the network congestion increases, natively balancing the trade-off between speed and sustainability.

6.1 Limitations

While the proposed off-chain batching middleware successfully demonstrated a carbon emission reduction on the Ethereum network, this study operates under several limitations.

The evaluation of the proposed middleware relies heavily on an isolated simulation environment, specifically the Hardhat node and the containerised VeChain Thor solo node. While the Ethereum network successfully forked the live mainnet state, it still functioned as a controlled sandbox. Consequently, the simulations do not fully account for real-world network anomalies, such as unpredictable block congestion, which would impact the execution success and latency of batched transactions on a live layer-one network.

In contrast, prior work such as iBatch by Wang et al. (2021) adopts a more realistic evaluation methodology by using a transaction replay framework based on real-world workload traces. In their approach, historical transaction data from deployed decentralised applications is collected using an instrumented Ethereum client and then replayed at scale using a high-throughput execution engine. This makes it possible to simulate long periods of realistic network activity while maintaining accurate gas usage patterns and workload characteristics. In addition, transaction inclusion and ordering are approximated using gas price based heuristics, providing a closer representation of real network behaviour.

The scope of the experimental framework was also restricted to a single fiat-backed stablecoin (USDC). Because USDC operates on standard ERC-20 and VIP-180 logic, its computational execution path remains relatively predictable, allowing for a more controlled and consistent evaluation of the proposed batching middleware. While this helps isolate the effects of the middleware, it also introduces a constraint in terms of generalisability. The observed gas savings and carbon emission reduction metrics may not directly extend to other tokens or blockchain networks that exhibit different execution characteristics, fee dynamics, or levels of network congestion. A more comprehensive evaluation would involve testing the middleware across a wider range of tokens and multiple blockchain environments, capturing a broader spectrum of real-world conditions. However, such an extension was beyond the practical scope and time constraints of this study.

Finally, the architectural design of the proposed middleware inherently introduces centralisation. By routing transactions through a single off-chain batching server, the system creates a single point of failure. On VeChain, this centralisation was further aggravated by the reliance on a singular god wallet to subsidise transaction fees. While this design is highly efficient for carbon emission reduction, it fundamentally conflicts with the decentralised characteristics of blockchain technology.

6.2 Future Work

The results of this study establish a solid baseline for transaction batching. However, several areas require further investigation before the system can be considered viable in real-world blockchain environments.

The current implementation relies on fixed parameters, which proved to be a limiting factor under volatile network conditions. In particular, the simulations demonstrated that small, static batch sizes lead to instability and latency spikes when transaction throughput increases. Future iterations should therefore focus on designing a dynamic control mechanism capable of continuously tuning both the batch size (N) and the batch interval (t) in response to real-time conditions. Such a system could leverage indicators such as mempool congestion and incoming transaction rates to maintain stability

while preserving carbon emission efficiency.

Another important avenue is the extension of this framework to Layer-2 scaling solutions. These networks operate under fundamentally different cost structures due to their rollup-based architectures, where computation and data availability are handled differently from Layer-1 systems. As a result, the relationship between batching, gas consumption, and carbon consumption may not follow the same patterns observed in this study. Evaluating the proposed middleware within these environments would provide a more comprehensive understanding of its effectiveness and could potentially reveal additional optimisation opportunities unique to Layer-2 ecosystems.

Furthermore, although this report provides a comparative baseline by testing the middleware on Ethereum (Proof of Stake) and VeChain (Proof of Authority), the results are limited to these specific architectures. Future work should extend this analysis to other major Layer-1 networks with different consensus mechanisms. Evaluating the proposed system, or similar off-chain batching approaches, across a broader range of networks is important to understand whether the observed carbon reduction benefits or potential system limitations are consistent or are mainly influenced by the characteristics of each network.

Finally, to address the centralisation limitation, subsequent iterations of this project should investigate methods to decentralise the off-chain middleware. Transitioning the batching coordinator from a single server application to a distributed relayer network would ensure the system retains its environmental efficiency without compromising the security, fault tolerance, and trustless nature of blockchain technology.

Bibliography

Alchemy Insights (n.d.), ‘Alchemy - the web3 development platform’.

URL: <https://www.alchemy.com>

Bajra, U. Q., Rogova, N. E. & Avdiaj, S. (2024), ‘Cryptocurrency blockchain and its carbon footprint: Anticipating future challenges’, *Technology in society* **77**, 102571–102571.

Bez, M., Fornari, G. & Vardanega, T. (2019), The scalability challenge of ethereum: An initial quantitative analysis, in ‘2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)’, pp. 167–176.

Binance Research (2026), ‘Full-year 2025 themes 2026’.

URL: <https://public.bnbstatic.com/static/files/research/full-year-2025-and-themes-for-2026.pdf>

Britz, K. (2019), ‘VIP-190: Personal Sign Standard’, Vechain Improvement Proposals (VIPs).

URL: <https://github.com/vechain/VIPs/blob/master/vips/VIP-190.md>

Buterin, V. (2016), ‘Ethereum: platform review’, *Opportunities and challenges for private and consortium blockchains* **45**, 1–45.

Buterin, V. et al. (2014), ‘Ethereum white paper’, Ethereum Foundation.

Cambridge Centre for Alternative Finance (n.d.), ‘Cambridge blockchain network sustainability index’, <https://ccaf.io/cbnsi/ethereum>. Accessed: 19 March 2026. Licensed under CC BY-NC-SA 4.0.

Chang, V. I., Ge, J., Hu, H. & Li, Z. (2023), ‘Lyapunov optimization - an overview — sciencedirect topics’.

URL: <https://www.sciencedirect.com/topics/computer-science/lyapunov-optimization>

Di Pierro, M. (2017), ‘What is the blockchain?’, *Computing in Science Engineering* **19**(5), 92–95.

Digiconomist (n.d.), ‘Api documentation - digiconomist’. Accessed: 8 November 2025.

URL: <https://digiconomist.net/api-documentation/>

Docker (2018), ‘Enterprise container platform — docker’.

URL: <https://www.docker.com>

EL Makhtoum, H. & Bentaleb, Y. (2022), Comparative study of keccak and blake2 hash functions, in M. Ben Ahmed, H.-N. L. Teodorescu, T. Mazri, P. Subashini & A. A. Boudhir, eds, ‘Networking, Intelligent Systems and Security’, Springer Singapore, Singapore, pp. 343–350.

Ethereum Energy Consumption — ethereum.org (2025).

URL: <https://ethereum.org/energy-consumption/>

ethereum.org (2025), ‘Proof-of-stake (pos) — ethereum.org’.

URL: <https://ethereum.org/developers/docs/consensus-mechanisms/pos/>

etherscan.io (2015), 'Ethereum daily gas used chart — etherscan'.

URL: <https://etherscan.io/chart/gasused>

Ethers.js Documentation (n.d.).

URL: <https://docs.ethers.org/v6/>

Gudgeon, L., Moreno-Sanchez, P., Roos, S., McCorry, P. & Gervais, A. (2020), Sok: Layer-two blockchain protocols, in 'International Conference on Financial Cryptography and Data Security', Springer, pp. 201–226.

Hashemi Joo, M., Nishikawa, Y. & Dandapani, K. (2019), 'Cryptocurrency, a successful application of blockchain technology', *Managerial Finance* **46**, 715–733.

Hassani, H., Huang, X. & Silva, E. (2018), 'Banking with blockchain-ed big data', *Journal of Management Analytics* **5**(4), 256–275.

URL: <https://doi.org/10.1080/23270012.2018.1528900>

Hossain, M. S. (2021), 'What do we know about cryptocurrency? past, present, future', *China Finance Review International* **ahead-of-print**.

URL: <https://www.emerald.com/insight/content/doi/10.1108/cfri-03-2020-0026/full/html>

Hui, R. (2023), 'Lightlink: The gasless blockchain network'.

URL: <https://lightlink-assets.s3.ap-southeast-1.amazonaws.com/docs/LightLink+Whitepaper+V2.pdf>

Härdle, W. K., Harvey, C. R. & Reule, R. C. G. (2020), 'Understanding cryptocurrencies*', *Journal of Financial Econometrics* **18**(2), 181–208.

URL: <https://doi.org/10.1093/jjfinec/nbz033>

Jia, J., Wu, O., Li, S., Ma, R. & Zhang, H. (2024), A scheduling algorithm for hyperledger fabric based on transaction batch processing, in '2024 23rd International Symposium on Parallel and Distributed Computing (ISPDC)', pp. 1–8.

Khan, M. M. A., Sarwar, H. M. A. & Awais, M. (2022), 'Gas consumption analysis of ethereum blockchain transactions', *Concurrency and Computation: Practice and Experience* **34**(4), e6679. e6679 CPE-20-1346.R2.

URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.6679>

Lashkari, B. & Musilek, P. (2021), 'A comprehensive review of blockchain consensus mechanisms', *IEEE access* **9**, 43620–43652.

Law, A. M., Kelton, W. D. & Kelton, W. D. (2007), *Simulation modeling and analysis*, Vol. 3, Mcgraw-hill New York.

Ma, F., Ren, M., Fu, Y., Wang, M., Li, H., Song, H. & Jiang, Y. (2021), 'Security reinforcement for ethereum virtual machine', *Information Processing Management* **58**(4), 102565.

URL: <https://www.sciencedirect.com/science/article/pii/S0306457321000674>

Madhwal, Y., Yanovich, Y., Korotkevich, A., Parshina, D., Seropian, N., Gavrilov, S., Nikolaev, A., Balachander, S. & Murugan, A. (2025), 'Empowering autonomous iot devices in blockchain through gasless transactions', *Blockchain: Research and Applications* **6**(2), 100257.

URL: <https://www.sciencedirect.com/science/article/pii/S2096720924000708>

Makarov, I., Schoar, A. et al. (2022), 'Cryptocurrencies and decentralized finance', *NBER Working Paper* **30006**.

Marin, O., Cioara, T., Todorean, L., Mitrea, D. & Anghel, I. (2023), 'Review of blockchain tokens creation and valuation', *Future Internet* **15**(12).

URL: <https://www.mdpi.com/1999-5903/15/12/382>

- Mita, M., Ito, K., Ohsawa, S. & Tanaka, H. (2019), What is stablecoin?: A survey on price stabilization mechanisms for decentralized payment systems, in '2019 8th International Congress on Advanced Applied Informatics (IIAI-AAI)', pp. 60–66.
- Mohanta, B. K., Panda, S. S. & Jena, D. (2018), An overview of smart contract and use cases in blockchain technology, in '2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)', pp. 1–4.
- Nomic Foundation (n.d.), 'Hardhat — ethereum development environment for professionals by nomic foundation'.
URL: <https://hardhat.org>
- Petrović, E. K., Brent, A. C., Iorns Magallanes, C., Hamer, L. & van Eijck, D. (2022), *Blockchain for Supply Chain Ledgers: Tracking Toxicity Information of Construction Materials*, Springer Nature Singapore, Singapore, pp. 89–111.
URL: https://doi.org/10.1007/978-981-19-3759-0_6
- Porkodi, S. & Kesavaraja, D. (2024), 'Escalating gas cost optimization in smart contract', *Wireless Personal Communications* **136**(1), 35–59.
URL: <https://doi.org/10.1007/s11277-024-11066-7>
- Rebello, G. A. F., Camilo, G. F., de Souza, L. A. C., Potop-Butucaru, M., de Amorim, M. D., Campista, M. E. M. & Costa, L. H. M. K. (2024), 'A survey on blockchain scalability: From hardware to layer-two protocols', *IEEE Communications Surveys Tutorials* **26**(4), 2411–2458.
- Sedlmeir, J., Buhl, H. U., Fridgen, G. & Keller, R. (2020), 'The energy consumption of blockchain technology: Beyond myth', *Business and Information Systems Engineering* **62**, 599–608.
- Sheffield, C. & Wijeyekoon, A. (2024), 'Gas less transaction on blockchain', Technical Disclosure Commons.
URL: https://www.tdcommons.org/dpubs_series/6791
- Swende, M. H. & Johnson, N. (2016), 'EIP-191: Signed Data Standard', Ethereum Improvement Proposals.
URL: <https://eips.ethereum.org/EIPS/eip-191>
- Ta, M. T. & Do, T. Q. (2024), 'A study on gas cost of ethereum smart contracts and performance of blockchain on simulation tool', *Peer-to-Peer Networking and Applications* **17**(1), 200–212.
URL: <https://doi.org/10.1007/s12083-023-01598-3>
- VeChain Foundation (2025a), 'Sdk — vechain docs'.
URL: <https://docs.vechain.org/developer-resources/sdks-and-providers/sdk>
- VeChain Foundation (2025b), 'Transaction calculation — vechain docs'.
URL: <https://docs.vechain.org/core-concepts/transactions/transaction-calculation>
- VeChain Fundation (2024), 'Consensus deep dive — vechain docs'.
URL: <https://docs.vechain.org/introduction-to-vechain/about-the-vechain-blockchain/consensus-deep-dive>
- VeChainStats (2026), 'Account - 0xc0c789a13a69859d3ae7bdb3fe4fa1625d20fd65'.
URL: <https://vechainstats.com/account/0xc0c789a13a69859d3ae7bdb3fe4fa1625d20fd65/contract-code>
- VeChainStats (n.d.), 'Vechain explorer and network statistics'. Accessed: 5 January 2026.
URL: <https://vechainstats.com>

Vercel (n.d.), 'Next.js by vercel - the react framework'.

URL: <https://nextjs.org>

Wang, N., Wang, B., Liu, T., Li, W. & Yang, S. (2020), A middleware approach to synchronize transaction data to blockchain, in '2020 29th International Conference on Computer Communications and Networks (ICCCN)', pp. 1–8.

Wang, Y., Zhang, Q., Li, K., Tang, Y., Chen, J., Luo, X. & Chen, T. (2021), 'ibatch: saving ethereum fees via secure and cost-effective batching of smart-contract invocations', *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.

Web3 for Better Whitepaper 3.0 (2023).

URL: <https://files.vechain.org/vechaincontainer/vechain-whitepaper-3-0.pdf>

Wood, G. et al. (2014), 'Ethereum: A secure decentralised generalised transaction ledger', *Ethereum project yellow paper* **151**(2014), 1–32.

Yang, R., Murray, T., Rimba, P. & Parampalli, U. (2019), 'Empirically analyzing ethereum's gas mechanism', *arXiv (Cornell University)*.

Zhang, R. & Chan, W. K. V. (2020), 'Evaluation of energy consumption in block-chains with proof of work and proof of stake', *Journal of Physics: Conference Series* **1584**(1), 012023.

Zhao, W. & Luo, X. (2025), Layer 2 blockchains: An introduction, in 'Proceedings of the 2024 7th Artificial Intelligence and Cloud Computing Conference', Association for Computing Machinery, New York, NY, USA, pp. 449–456.

URL: <https://doi.org/10.1145/3719384.3719451>

Zohuri, B., Nguyen, H. & Moghaddam, M. (2022), 'What is the cryptocurrency? is it a threat to our national security, domestically and globally?', *AICyberDomain.com* **3**, 1–14.

URL: <https://unisciencepub.com/wp-content/uploads/2022/02/What-is-the-Cryptocurrency-Is-it-a-Threat-to-Our-National-Security-Domestically-and-Globally.pdf>

Appendix A

Smart Contract Source Code

This chapter presents the full Solidity source code for the on-chain execution layer of the proposed middleware. These contracts serve as the secure interface between the off-chain batcher and the blockchain networks. By implementing custom logic to handle multi-sender transfers, these contracts enable the system to verify individual user authorizations through cryptographic signatures before executing aggregated transactions in a single batch.

A.1 Ethereum Batcher Smart Contract

This section presents the `ETHBatch` Solidity contract used to implement the on-chain portion of the middleware for the Ethereum network. The code defines the logic for receiving parallel arrays of transaction data and utilizing the `ecrecover` function to verify cryptographic signatures against EIP-191 standards. Once authenticated, the contract executes the `transferFrom` function on the USDC smart contract on behalf of the original senders.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 interface IERC20 {
5     function transferFrom(
6         address sender,
7         address recipient,
8         uint256 amount
9     ) external returns (bool);
10 }
11
12 contract ETHBatch {
13     address constant USDC_ADDRESS = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;
14
15     mapping(address => uint256) public nonces;
16
17     function executeBatch(
18         address[] calldata senders,
19         address[] calldata recipients,
20         uint256[] calldata amounts,
21         bytes[] calldata signatures
22     ) external {
23         //call sender.length just once to save gas
24         uint256 len = senders.length;
25         require(
26             recipients.length == len && amounts.length == len,
27             "Array mismatch"
28         );
29
30         for (uint256 i = 0; i < len; ) {
```

```

31     _verifyAndTransfer(
32         senders[i],
33         recipients[i],
34         amounts[i],
35         signatures[i]
36     );
37
38     unchecked {
39         i++;
40     } //this is to optimise gas for loops
41 }
42
43
44 function _verifyAndTransfer(
45     address sender,
46     address recipient,
47     uint256 amount,
48     bytes calldata signature
49 ) internal {
50     uint256 currentNonce = nonces[sender];
51
52     bytes32 messageHash = keccak256(
53         abi.encodePacked(sender, recipient, amount, currentNonce)
54     );
55
56     //EIP-191 Prefix
57     bytes32 ethHash = keccak256(
58         abi.encodePacked("\x19Ethereum Signed Message:\n32", messageHash)
59     );
60
61     nonces[sender] = currentNonce + 1;
62
63     require(signature.length == 65, "Invalid signature length");
64
65     bytes32 r;
66     bytes32 s;
67     uint8 v;
68     assembly {
69         r := calldataload(signature.offset)
70         s := calldataload(add(signature.offset, 32))
71         v := byte(0, calldataload(add(signature.offset, 64)))
72     }
73
74     //recover and verify
75     address recoveredSigner = ecrecover(ethHash, v, r, s);
76     require(
77         recoveredSigner != address(0) && recoveredSigner == sender,
78         "Invalid signature"
79     );
80
81     require(
82         IERC20(USDC_ADDRESS).transferFrom(sender, recipient, amount),
83         "Transfer failed"
84     );
85 }
86 }

```

Listing A.1: Batchers smart contract for the Ethereum network

A.2 VeChain Batcher Smart Contract

This section contains the source code for the `VeChainBatch` contract deployed on the `VeChainThor` solo node. The implementation includes logic for reconstructing message hashes using `Blake2b256` and the `VIP-190` signing standard. The system is designed to process signatures that have been manually adjusted at the off-chain layer to ensure the `v` value remains compatible with the Solidity `ecrecover` execution environment.

```
1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.28;
4
5 interface IVIP180 {
6     function transferFrom(
7         address sender,
8         address recipient,
9         uint256 amount
10    ) external returns (bool);
11 }
12
13 interface IExtension {
14     function blake2b256(bytes memory data) external view returns (bytes32);
15 }
16
17 contract VeChainBatch {
18     IVIP180 public immutable targetToken;
19     address constant EXTENSION_ADDR =
20         0x0000000000000000000000000000000000000000000000000000000000000000;
21     mapping(address => uint256) public nonces;
22
23     constructor(address usdcAddress) {
24         targetToken = IVIP180(usdcAddress);
25     }
26
27     function executeBatch(
28         address[] calldata senders,
29         address[] calldata recipients,
30         uint256[] calldata amounts,
31         bytes[] calldata signatures
32    ) external {
33         uint256 len = senders.length;
34         require(
35             recipients.length == len && amounts.length == len,
36             "Array mismatch"
37         );
38
39         for (uint256 i = 0; i < len; ) {
40             _verifyAndTransfer(
41                 senders[i],
42                 recipients[i],
43                 amounts[i],
44                 signatures[i]
45             );
46
47             unchecked {
48                 i++;
49             } //this is to optimise gas for loops
50         }
51     }
52
53     function _verifyAndTransfer(
54         address sender,
55         address recipient,
```

```

56     uint256 amount,
57     bytes calldata signature
58 ) internal {
59     require(signature.length == 65, "Invalid signature length");
60     uint256 currentNonce = nonces[sender];
61
62     bytes memory rawMessage = abi.encodePacked(
63         sender,
64         recipient,
65         amount,
66         currentNonce
67     );
68     bytes32 messageHash = IExtension(EXTENSION_ADDR).blake2b256(rawMessage);
69
70     nonces[sender] = currentNonce + 1;
71
72     bytes32 r;
73     bytes32 s;
74     uint8 v;
75     assembly {
76         r := calldataload(signature.offset)
77         s := calldataload(add(signature.offset, 32))
78         v := byte(0, calldataload(add(signature.offset, 64)))
79     }
80
81     //recover and verify
82     address recoveredSigner = ecrecover(messageHash, v, r, s);
83     require(
84         recoveredSigner != address(0) && recoveredSigner == sender,
85         "Invalid signature"
86     );
87
88     require(
89         targetToken.transferFrom(sender, recipient, amount),
90         "Transfer failed"
91     );
92 }
93 }

```

Listing A.2: Batcher smart contract for the VeChain network

Appendix B

Simulation Source Code

This appendix presents the full TypeScript source code for the off-chain simulation engine and middleware layer. These scripts serve as the central control unit for the research, managing the end-to-end experimental flow by generating synthetic transaction loads, coordinating with the on-chain smart contracts, and compiling raw execution results into structured JSON logs.

The implementation details the distinct logic required to bridge Ethereum's immediate transaction processing with VeChain's deterministic 10-second block schedule. Furthermore, this section includes the algorithmic implementation of the Inverse-Transform Sampling method used to model realistic, Poisson-distributed network traffic during the two-hour experimental runs.

B.1 Ethereum Simulation Source Code

This script implements the simulation engine specifically for the Ethereum network. It utilizes the Ethers.js library to manage all cryptographic operations and interactions with the local Hardhat development node. The code demonstrates how the middleware executes standard ERC-20 transfers individually to establish a baseline before aggregating transactions into the batching smart contract using EIP-191 message signing.

```
1
2 import { generateRandomTransaction } from "../lib/generateRandomUSDCTransaction";
3 import { adminWallet } from "../lib/ethereum-wallets";
4 import { ethers } from "ethers";
5 import type {
6   IndividualTxLog,
7   SimulationLog,
8   Transaction,
9 } from "../types/types";
10 import { ETH_BATCH_CONTRACT_ABI } from "../lib/ABI";
11 import * as dotenv from "dotenv";
12 import { saveLog } from "@lib/saveLog";
13
14 dotenv.config();
15
16 const HARDHAT_RPC_URL = "http://127.0.0.1:8545";
17 const USDC_ADDRESS = "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48";
18 const BATCH_CONTRACT_ADDRESS = process.env
19   .NEXT_PUBLIC_ETHEREUM_BATCHER_ADDRESS as `0x${string}`;
20 const USDC_ABI = [
21   {
22     name: "transfer",
23     type: "function",
24     stateMutability: "nonpayable",
25     inputs: [
26       { name: "to", type: "address", internalType: "address" },
27       { name: "amount", type: "uint256", internalType: "uint256" },
28     ],
29   },
30 ];
```

```

28     ],
29     outputs: [{ name: "", type: "bool", internalType: "bool" }],
30   },
31 ] as const;
32
33 //batching variables
34 const BATCH_SIZE = Number(process.env.NEXT_PUBLIC_SIMULATION_BATCH_SIZE);
35 const BATCH_INTERVAL_MIN = Number(
36   process.env.NEXT_PUBLIC_SIMULATION_BATCH_INTERVAL_MIN,
37 );
38 const BATCH_INTERVAL_MS = BATCH_INTERVAL_MIN * 60 * 1000;
39 const SIMULATION_DURATION_MIN = Number(
40   process.env.NEXT_PUBLIC_SIMULATION_DURATION_MIN,
41 );
42 const SIMULATION_DURATION = SIMULATION_DURATION_MIN * 60 * 1000;
43
44 const TARGET_THROUGHPUT = Number(process.env.NEXT_PUBLIC_TARGET_TPS || 5);
45
46 const simulationLog: SimulationLog = {
47   simulationStartTime: Date.now(),
48   simulationEndTime: 0,
49   simulationDuration: SIMULATION_DURATION,
50   batchSize: BATCH_SIZE,
51   batchIntervalMinutes: BATCH_INTERVAL_MIN,
52   throughput: TARGET_THROUGHPUT,
53   individualTransactions: [],
54   batches: [],
55   summary: {
56     totalIndividualTransactions: 0,
57     totalBatches: 0,
58     totalIndividualGasUsed: "0",
59     totalBatchGasUsed: "0",
60   },
61 };
62
63 let individualTransactionsBuffer: IndividualTxLog[] = [];
64
65 function getPoissonDelay(targetTPS: number): number {
66   //prevent division by zero
67   if (targetTPS <= 0) return 3000;
68
69   const ratePerMs = targetTPS / 1000;
70
71   //The formula: -ln(U) /
72   const delayMs = -Math.log(Math.random()) / ratePerMs;
73
74   return delayMs;
75 }
76
77 async function executeBatch(
78   batch: Transaction[],
79   batcherWallet: ethers.Wallet,
80   batchNumber: number,
81   provider: ethers.JsonRpcProvider,
82 ) {
83   if (batch.length === 0) {
84     console.log(
85       '\n Batch #${batchNumber}: No transactions to batch. Skipping...\n',
86     );
87     return;
88   }
89
90   if (batch.length >= BATCH_SIZE) {

```

```

91     console.log(`Batch size is reached.`);
92 }
93
94 console.log(
95     `\n Batch #${batchNumber}: Batching ${batch.length} transactions...`,
96 );
97
98 const contract = new ethers.Contract(
99     BATCH_CONTRACT_ADDRESS,
100     ETH_BATCH_CONTRACT_ABI,
101     batcherWallet,
102 );
103
104 try {
105     const signatures: string[] = [];
106     const senders = [];
107     const recipients = [];
108     const amounts = [];
109
110     const senderNoncesMap = new Map<string, bigint>();
111     const batchSnapshot = [...batch]
112
113     //every sender needs to sign the transaction to be included in the batch
114     for (let i = 0; i < batchSnapshot.length; i++) {
115         const tx = batchSnapshot[i];
116
117         const senderWallet = new ethers.Wallet(
118             tx.senderPrivateKey as string,
119             provider,
120         );
121
122         let nonce: bigint;
123
124         if (senderNoncesMap.has(tx.sender)) {
125             nonce = senderNoncesMap.get(tx.sender)!;
126             senderNoncesMap.set(tx.sender, nonce + BigInt(1));
127         } else {
128             nonce = await contract.nonces(tx.sender);
129             senderNoncesMap.set(tx.sender, nonce + BigInt(1));
130         }
131
132         const messageHash = ethers.solidityPackedKeccak256(
133             ["address", "address", "uint256", "uint256"],
134             [tx.sender, tx.recipient, tx.amount, nonce],
135         );
136
137         const signature = await senderWallet.signMessage(
138             ethers.getBytes(messageHash),
139         );
140
141         signatures.push(signature);
142         senders.push(tx.sender);
143         recipients.push(tx.recipient);
144         amounts.push(tx.amount);
145     }
146
147     const batchedTx = await contract.executeBatch(
148         senders,
149         recipients,
150         amounts,
151         signatures,
152     );
153

```

```

154 console.log(`\nBatch #${batchNumber} Tx Sent: ${batchedTx.hash}`);
155
156 const batchedTxReceipt = await batchedTx.wait();
157
158 const batchGasUsed =
159     batchedTxReceipt &&
160     (typeof batchedTxReceipt.gasUsed === "bigint"
161         ? batchedTxReceipt.gasUsed.toString()
162         : String(batchedTxReceipt.gasUsed));
163
164 console.log(
165     '\n Batch #${batchNumber} Confirmed! Total Gas: ${batchGasUsed}',
166 );
167
168 console.log("\n-----");
169
170 //add batch to the log
171
172 simulationLog.batches.push({
173     batchNumber,
174     gasUsed: batchGasUsed,
175     timestamp: Date.now(),
176     transactionCount: batch.length,
177     transactions: batch.map((tx) => ({
178         sender: tx.sender,
179         recipient: tx.recipient,
180         amount: tx.amount.toString(),
181         timeStamp: tx.timeStamp,
182     })),
183 });
184
185 //data of individual transactions wont be added to the dataset until the batch
186 //is executed without any issues.
187 simulationLog.individualTransactions.push(...individualTransactionsBuffer);
188 individualTransactionsBuffer = [];
189 } catch (error) {
190     console.error(` Batch #${batchNumber} execution failed:`, error);
191 }
192
193 async function USDCSimulation() {
194     const provider = new ethers.JsonRpcProvider(HARDHAT_RPC_URL);
195
196     console.log("Starting Background Worker...");
197     console.log(
198         ` Simulation Duration: ${SIMULATION_DURATION / 1000 / 60} minutes`,
199     );
200     console.log(` Batch Interval: Every ${BATCH_INTERVAL_MIN} minutes\n`);
201     console.log(`Batch Size: ${BATCH_SIZE}`);
202     console.log(` Throughput: ${TARGET_THROUGHPUT} per second`);
203
204     const batcherWallet = new ethers.Wallet(adminWallet.privateKey, provider);
205
206     let activeProcesses = 0;
207
208     const startTime = Date.now();
209     const endTime = startTime + SIMULATION_DURATION;
210     let nextBatchTime = startTime + BATCH_INTERVAL_MS;
211     let batch: Transaction[] = [];
212     let batchNumber = 1;
213
214     // Send countdown updates
215     const countdownInterval = setInterval(() => {

```

```

216     const remaining = Math.ceil((endTime - Date.now()) / 1000);
217     const nextBatchIn = Math.ceil((nextBatchTime - Date.now()) / 1000);
218     if (remaining > 0) {
219         process.stdout.write(
220             '\r Total remaining: ${remaining}s | Next batch in: ${nextBatchIn}s |
Collected: ${batch.length} txs ',
221         );
222     } else {
223         process.stdout.write(
224             '\r Time Window Closed.                                \n',
225         );
226         clearInterval(countdownInterval);
227     }
228 }, 1000);
229
230 const processNewTransaction = async () => {
231     activeProcesses++;
232     try {
233         const transaction = await generateRandomTransaction();
234
235         const recipient = transaction.recipient;
236         const txamount = transaction.amount;
237         const individualWallet = new ethers.Wallet(
238             transaction.senderPrivateKey as string,
239             provider,
240         );
241         const individualUsdc = new ethers.Contract(
242             USDC_ADDRESS,
243             USDC_ABI,
244             individualWallet,
245         );
246
247         const tx = await individualUsdc.transfer(recipient, txamount);
248         const txReceipt = await tx.wait();
249
250         const gasUsed =
251             txReceipt &&
252             (typeof txReceipt.gasUsed === "bigint"
253                 ? txReceipt.gasUsed.toString()
254                 : String(txReceipt.gasUsed));
255
256         console.log('\n Individual Tx: ${tx.hash}');
257         console.log(` Gas Used: ${gasUsed}`);
258
259         console.log("-----\n");
260
261         //add the transaction to the log, if it fails it wont be added
262         //add them to the buffer first. if the batch fails, we wont add these
transactions to the data.
263         individualTransactionsBuffer.push({
264             sender: transaction.sender,
265             recipient: transaction.recipient,
266             amount: transaction.amount.toString(),
267             gasUsed: gasUsed,
268             timestamp: transaction.timeStamp,
269         });
270
271         batch.push(transaction);
272     } catch (txError) {
273         console.error("Transaction failed:", txError);
274     } finally {
275         activeProcesses--;
276     }

```

```

277 };
278
279 try {
280     while (Date.now() < endTime) {
281         // Check if it's time to execute a batch
282         if (Date.now() >= nextBatchTime || batch.length >= BATCH_SIZE) {
283             nextBatchTime = Date.now() + BATCH_INTERVAL_MS;
284             await executeBatch(batch, batcherWallet, batchNumber, provider);
285             batch = []; // Clear the batch
286             batchNumber++;
287         }
288
289         processNewTransaction();
290
291         //manage throughput
292         await new Promise((r) =>
293             setTimeout(r, getPoissonDelay(TARGET_THROUGHPUT)),
294         );
295     }
296
297     //wait for all pending transactions to be executed
298     console.log("\nWaiting for pending transactions...");
299     while (activeProcesses > 0) {
300         await new Promise((r) => setTimeout(r, 1000));
301     }
302
303     // Execute any remaining transactions in the batch after simulation ends
304     if (batch.length > 0) {
305         console.log("Executing final batch with remaining transactions...");
306         await executeBatch(batch, batcherWallet, batchNumber, provider);
307     }
308
309     console.log(`\n--- Simulation Complete ---\n`);
310
311     simulationLog.simulationEndTime = Date.now();
312     saveLog(simulationLog, "simulation/EthereumSimulationLogs");
313 } catch (error) {
314     console.error(" FATAL ERROR:", error);
315     simulationLog.simulationEndTime = Date.now();
316     saveLog(simulationLog, "simulation/EthereumSimulationLogs");
317 } finally {
318     clearInterval(countdownInterval);
319 }
320 }
321
322 USDCSimulation()
323     .then(() => process.exit(0))
324     .catch((e) => {
325         console.error(e);
326         process.exit(1);
327     });

```

Listing B.1: TypeScript simulation script for the Ethereum network

B.2 VeChain Simulation Source Code

This section details the simulation logic designed for the VeChain Thor network. Unlike the Ethereum implementation, this script relies on a hybrid approach, using the VeChain SDK to handle the network's unique transaction model and Blake2b hashing standards. The code is specifically architected to accommodate VeChain's deterministic 10-second block generation schedule, utilizing asynchronous submission and bulk receipt retrieval to prevent the middleware from overwhelming the mempool. It

also includes the necessary signature formatting adjustments to ensure the VIP-190 signatures remain compatible with the Ethereum Virtual Machine (EVM) architecture used by VeChain's smart contracts.

```
1
2 import {
3   VECHAIN_BATCH_CONTRACT_ABI,
4   VECHAIN_USDC_CONTRACT_ABI,
5 } from "@lib/ABI";
6 import {
7   IndividualTxLog,
8   SimulationLog,
9   Transaction as TransactionType,
10 } from "@types/types";
11 import {
12   ABIContract,
13   Address,
14   Clause,
15   Hex,
16   Transaction,
17   Mnemonic,
18   Secp256k1,
19   TransactionBody,
20   Blake2b256,
21 } from "@vechain/sdk-core";
22 import { ThorClient } from "@vechain/sdk-network";
23 import * as dotenv from "dotenv";
24 import { ethers } from "ethers";
25 import { generateRandomVeChainTransaction } from "@lib/
    generateRandomUSDCTransaction";
26 import { saveLog } from "@lib/saveLog";
27
28 dotenv.config();
29
30 //batching variables
31 const BATCH_SIZE = Number(process.env.NEXT_PUBLIC_SIMULATION_BATCH_SIZE);
32 const BATCH_INTERVAL_MIN = Number(
33   process.env.NEXT_PUBLIC_SIMULATION_BATCH_INTERVAL_MIN,
34 );
35 const BATCH_INTERVAL_MS = BATCH_INTERVAL_MIN * 60 * 1000;
36 const SIMULATION_DURATION_MIN = Number(
37   process.env.NEXT_PUBLIC_SIMULATION_DURATION_MIN,
38 );
39 const SIMULATION_DURATION = SIMULATION_DURATION_MIN * 60 * 1000;
40
41 const THOR_URL = "http://127.0.0.1:8669";
42 const USDC_ADDRESS = process.env.NEXT_PUBLIC_VECHAIN_USDC_ADDRESS as string;
43 const BATCHER_ADDRESS = process.env
44   .NEXT_PUBLIC_VECHAIN_BATCHER_ADDRESS as string;
45
46 const thorSoloClient = ThorClient.at(THOR_URL, {
47   isPollingEnabled: false,
48 });
49
50 const batchContract = thorSoloClient.contracts.load(
51   BATCHER_ADDRESS,
52   VECHAIN_BATCH_CONTRACT_ABI,
53 );
54
55 const godMnemonic =
56   "denial kitchen pet squirrel other broom bar gas better priority spoil cross";
57 const godPrivateKey = Mnemonic.toPrivateKey(godMnemonic.split(" "));
58
59 const TARGET_THROUGHPUT = Number(process.env.NEXT_PUBLIC_TARGET_TPS || 5);
60
```

```

61 const simulationLog: SimulationLog = {
62   simulationStartTime: Date.now(),
63   simulationEndTime: 0,
64   simulationDuration: SIMULATION_DURATION,
65   batchSize: BATCH_SIZE,
66   batchIntervalMinutes: BATCH_INTERVAL_MIN,
67   throughput: TARGET_THROUGHPUT,
68   individualTransactions: [],
69   batches: [],
70   summary: {
71     totalIndividualTransactions: 0,
72     totalBatches: 0,
73     totalIndividualGasUsed: "0",
74     totalBatchGasUsed: "0",
75   },
76 };
77
78 let individualTransactionsBuffer: IndividualTxLog[] = [];
79
80 function getPoissonDelay(targetTPS: number): number {
81   //prevent division by zero
82   if (targetTPS <= 0) return 3000;
83
84   const ratePerMs = targetTPS / 1000;
85
86   //The formula: -ln(U) /
87   const delayMs = -Math.log(Math.random()) / ratePerMs;
88
89   return delayMs;
90 }
91
92 async function executeBatch(batch: TransactionType[], batchNumber: number) {
93   if (batch.length === 0) {
94     console.log(
95       '\n Batch #${batchNumber}: No transactions to batch. Skipping...',
96     );
97     return;
98   }
99
100   if (batch.length >= BATCH_SIZE) {
101     console.log('\nBatch size is reached.');

```



```

124     if (senderNoncesMap.has(tx.sender)) {
125         nonce = senderNoncesMap.get(tx.sender)!;
126         senderNoncesMap.set(tx.sender, nonce + BigInt(1));
127     } else {
128         const nonceResult = await batchContract.read.nonces(tx.sender);
129         nonce = nonceResult[0];
130         senderNoncesMap.set(tx.sender, nonce + BigInt(1));
131     }
132
133     const packedData = ethers.solidityPacked(
134         ["address", "address", "uint256", "uint256"],
135         [tx.sender, tx.recipient, tx.amount, nonce],
136     );
137
138     const rawBytes = ethers.getBytes(packedData);
139
140     const messageHash = Blake2b256.of(rawBytes);
141
142     const signature = Secp256k1.sign(
143         messageHash.bytes,
144         ethers.getBytes(tx.senderPrivateKey!),
145     );
146
147     // Adjust recovery ID from raw Secp256k1 format (0/1) to Ethereum ecrecover
148     // format (27/28)
149     signature[64] += 27;
150
151     signatures.push(Hex.of(signature).toString());
152     senders.push(tx.sender);
153     recipients.push(tx.recipient);
154     amounts.push(tx.amount);
155 }
156
157 const clause: Clause = Clause.callFunction(
158     Address.of(BATCHER_ADDRESS),
159     ABIContract.ofAbi(VECHAIN_BATCH_CONTRACT_ABI).getFunction("executeBatch"),
160     [senders, recipients, amounts, signatures],
161 );
162
163 const batchTransaction = {
164     clauses: [clause],
165 };
166
167 const batchGasEstimateGas = await thorSoloClient.gas.estimateGas(
168     batchTransaction.clauses,
169 );
170
171 const batchTxBody = await thorSoloClient.transactions.buildTransactionBody(
172     batchTransaction.clauses,
173     batchGasEstimateGas.totalGas,
174 );
175
176 const signedTransaction = Transaction.of(batchTxBody).sign(godPrivateKey);
177
178 const sendTransactionResult =
179     await thorSoloClient.transactions.sendTransaction(signedTransaction);
180
181 const txReceipt = await thorSoloClient.transactions.waitForTransaction(
182     sendTransactionResult.id,
183 );
184
185 const batchGasUsed = String(txReceipt?.gasUsed);

```

```

186 console.log(txReceipt);
187
188 simulationLog.batches.push({
189   batchNumber,
190   gasUsed: batchGasUsed,
191   timestamp: Date.now(),
192   transactionCount: batch.length,
193   transactions: batch.map((tx) => ({
194     sender: tx.sender,
195     recipient: tx.recipient,
196     amount: tx.amount.toString(),
197     timeStamp: tx.timeStamp,
198   })),
199 });
200
201 simulationLog.individualTransactions.push(...individualTransactionsBuffer);
202 individualTransactionsBuffer = [];
203 } catch (error) {
204   console.error(` Batch #${batchNumber} execution failed`, error);
205 }
206 }
207
208 async function VeChainUSDCSimulation() {
209   console.log("Starting Background Worker...");
210
211   console.log(
212     `Simulation Duration: ${SIMULATION_DURATION / 1000 / 60} minutes`,
213   );
214   console.log(` Batch Interval: Every ${BATCH_INTERVAL_MIN} minutes\n`);
215   console.log(`Batch Size: ${BATCH_SIZE}`);
216   console.log(` Throughput: ${TARGET_THROUGHPUT} per second`);
217
218   let activeProcesses = 0; // track active promises
219
220   const startTime = Date.now();
221   const endTime = startTime + SIMULATION_DURATION;
222   let nextBatchTime = startTime + BATCH_INTERVAL_MS;
223   let batch: TransactionType[] = [];
224   let batchNumber = 1;
225
226   const countdownInterval = setInterval(() => {
227     const remaining = Math.ceil((endTime - Date.now()) / 1000);
228     const nextBatchIn = Math.ceil((nextBatchTime - Date.now()) / 1000);
229     if (remaining > 0) {
230       process.stdout.write(
231         `\r Total remaining: ${remaining}s | Next batch in: ${nextBatchIn}s |
232         Collected: ${batch.length} txs `,
233       );
234     } else {
235       process.stdout.write(`\r Time Window Closed.\n`);
236       clearInterval(countdownInterval);
237     }
238   }, 1000);
239
240   const processNewTransaction = async (chainTag: number) => {
241     activeProcesses++;
242     try {
243       const bestBlock = await thorSoloClient.blocks.getBestBlockCompressed();
244       const blockRef = bestBlock !== null ? bestBlock.id.slice(0, 18) : "0x0";
245
246       const transaction = await generateRandomVeChainTransaction();
247
248       const individualWallet = {

```

```

248     privateKey: transaction.senderPrivateKey,
249     address: transaction.sender,
250   };
251   const clauses: Clause[] = [
252     Clause.callFunction(
253       Address.of(USDC_ADDRESS),
254       ABISContract.ofAbi(VECHAIN_USDC_CONTRACT_ABI).getFunction("transfer"),
255       [transaction.recipient, transaction.amount / BigInt(1000000)],
256     ),
257   ];
258
259   const gas = await thorSoloClient.gas.estimateGas(
260     clauses,
261     individualWallet.address,
262   );
263
264   const body: TransactionBody = {
265     chainTag,
266     blockRef,
267     expiration: 2 ** 32 - 1,
268     clauses,
269     gasPriceCoef: 0,
270     gas: gas.totalGas,
271     dependsOn: null,
272     nonce: Date.now(),
273   };
274
275   const signedTransaction = Transaction.of(body).sign(
276     Hex.of(transaction.senderPrivateKey!).bytes,
277   );
278
279   const sendTransactionResult =
280     await thorSoloClient.transactions.sendTransaction(signedTransaction);
281
282   console.log("-----");
283
284   console.log(`Tx Sent (Pending in Mempool)`);
285
286   console.log("-----");
287
288   const txReceipt = await thorSoloClient.transactions.waitForTransaction(
289     sendTransactionResult.id,
290   );
291
292   const gasUsed = String(txReceipt!.gasUsed);
293
294   console.log(" Individual Tx executed");
295
296   individualTransactionsBuffer.push({
297     sender: transaction.sender,
298     recipient: transaction.recipient,
299     amount: transaction.amount.toString(),
300     gasUsed: gasUsed,
301     timestamp: transaction.timeStamp,
302   });
303
304   batch.push(transaction);
305 } catch (txError) {
306   console.error("Transaction failed:", txError);
307 } finally {
308   activeProcesses--;
309 }
310 };

```

```

311
312 try {
313     const chainTag = await thorSoloClient.nodes.getChaintag();
314     while (Date.now() < endTime) {
315         // Check if it's time to execute a batch
316         if (Date.now() >= nextBatchTime || batch.length >= BATCH_SIZE) {
317             nextBatchTime = Date.now() + BATCH_INTERVAL_MS;
318
319             const currentBatch = [...batch];
320             batch = []; // Clear the batch
321             await executeBatch(currentBatch, batchNumber);
322             batchNumber++;
323         }
324
325         processNewTransaction(chainTag);
326
327         //manage throughput
328         await new Promise((r) =>
329             setTimeout(r, getPoissonDelay(TARGET_THROUGHPUT)),
330         );
331     }
332
333     //wait for all pending transactions to be executed
334     console.log("\nWaiting for pending transactions...");
335     while (activeProcesses > 0) {
336         await new Promise((r) => setTimeout(r, 1000));
337     }
338
339     // Execute any remaining transactions in the batch after simulation ends
340     if (batch.length > 0) {
341         console.log("\nExecuting final batch with remaining transactions...");
342         await executeBatch(batch, batchNumber);
343     }
344
345     console.log('\n--- Simulation Complete ---\n');
346
347     simulationLog.simulationEndTime = Date.now();
348     saveLog(simulationLog, "simulation/VeChainSimulationLogs");
349 } catch (error) {
350     console.error("FATAL ERROR:", error);
351     simulationLog.simulationEndTime = Date.now();
352     saveLog(simulationLog, "simulation/VeChainSimulationLogs");
353 } finally {
354     clearInterval(countdownInterval);
355 }
356 }
357
358 VeChainUSDCSimulation()
359     .then(() => process.exit(0))
360     .catch((e) => {
361         console.error(e);
362         process.exit(1);
363     });

```

Listing B.2: TypeScript simulation script for the VeChain network

Appendix C

Extended Data Analysis

This appendix contains supplemental visualizations from the Ethereum and VeChain simulation runs. These figures provide additional granularity regarding transaction latency and carbon consumption across the various throughput levels and batch sizes tested in this research. They serve to further validate the results and evaluation presented in Chapter 5.

C.1 Ethereum Performance Visualisations

This section provides additional graphical data from the Ethereum (Proof of Stake) simulation runs. These visualizations offer further detail into the correlation between batch size and carbon reduction across varying throughput levels. The data points included here support the primary findings that a batch size of 150 yields the most significant environmental benefits on the Ethereum network.

Figure C.1 shows the line graphs to illustrate the linear relationship between average transaction latency and batch size across throughput levels of 2, 5, and 9 for the Ethereum network.

Figure C.2 is a scatter plot that illustrates the Pareto front representing the trade-off between average transaction latency and absolute CO₂ savings (in kg) across different throughput levels and batch sizes for the Ethereum network.

Figure C.3 is a bar chart comparing the absolute CO₂ emissions (in kg) of the unbatched baseline transactions versus the optimised batching system at a batch size of 150 across throughput levels of 2, 5, and 9 for the Ethereum network.

Figure C.4 presents line graphs illustrating the relationship between carbon emission reduction percentages and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the Ethereum network.

C.2 VeChain Performance Visualisations

This section includes supplemental graphs from the VeChain (Proof of Authority) simulation environment. These figures illustrate the transaction behavior and latency anomalies observed during testing, particularly the initial spikes occurring at the smallest batch sizes. The data further documents the inverse relationship between the proposed middleware and carbon efficiency on the VeChain network.

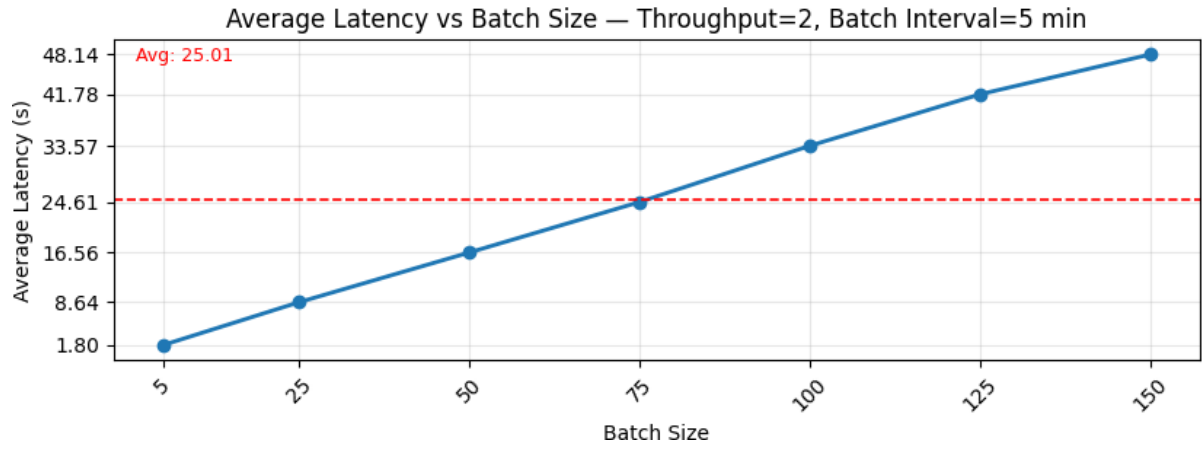
Figure C.5 presents line graphs illustrating the relationship between absolute CO₂ savings (which are negative, indicating an increase in emissions) and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

Figure C.6 is a violin plot that illustrates the transaction latency distribution by batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

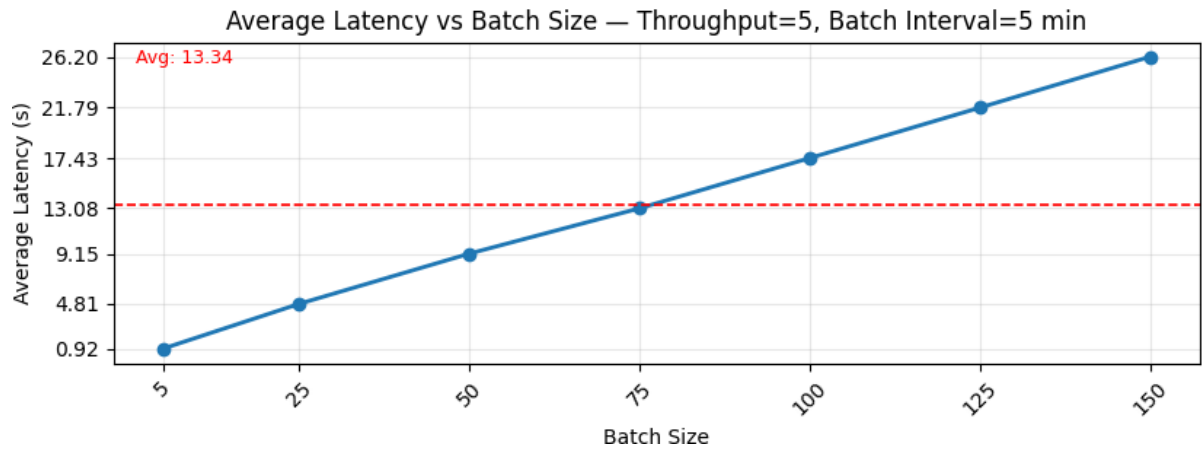
Figure C.7 is a scatter plot that illustrates the Pareto front representing the trade-off between average transaction latency and absolute CO₂ savings across different throughput levels and batch sizes for the VeChain network.

Figure C.8 presents line graphs illustrating the relationship between carbon emission reduction percentages and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

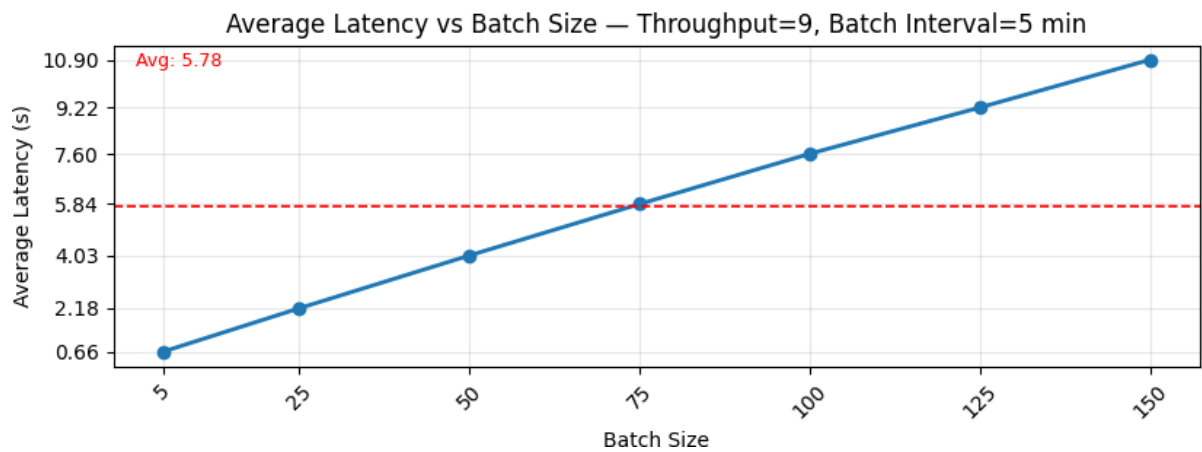
Figure C.9 is a bar chart comparing the CO_2 emissions of the unbatched baseline transactions against the optimised batching system across throughput levels of 2, 5, and 9 for the VeChain network.



(a) Throughput = 2



(b) Throughput = 5



(c) Throughput = 9

Figure C.1: Line graphs illustrating the relationship between average transaction latency and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the Ethereum network.

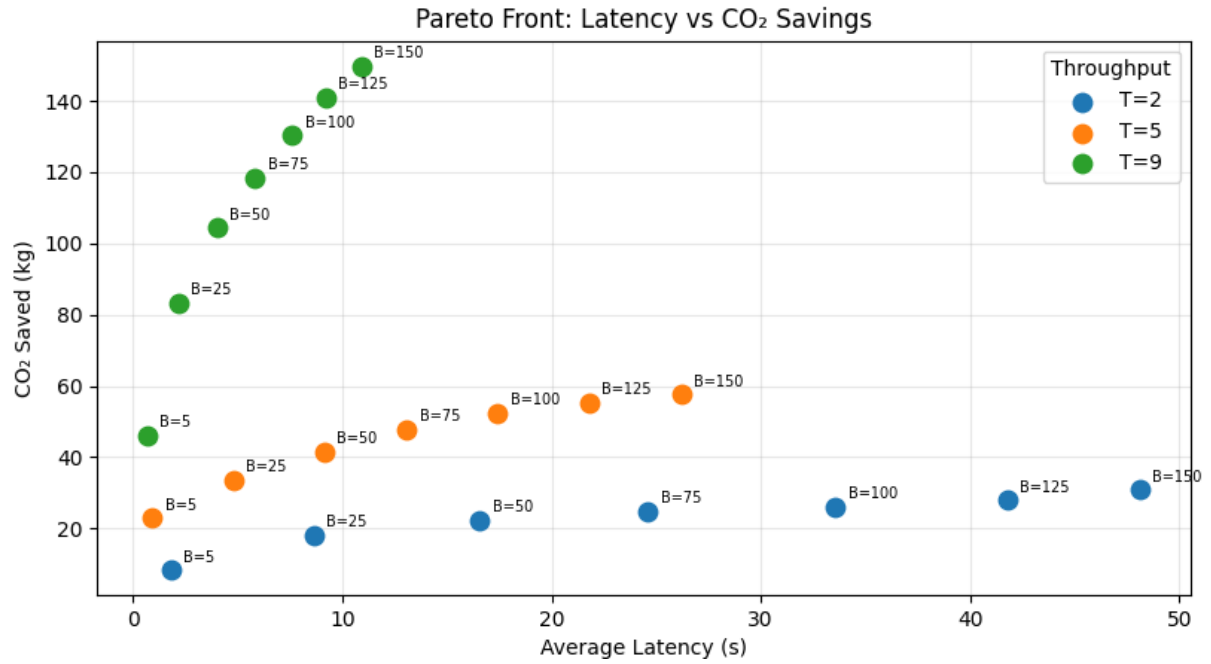


Figure C.2: Pareto front scatter plot showing the trade-off between average latency and absolute CO₂ savings (kg) for the Ethereum network.

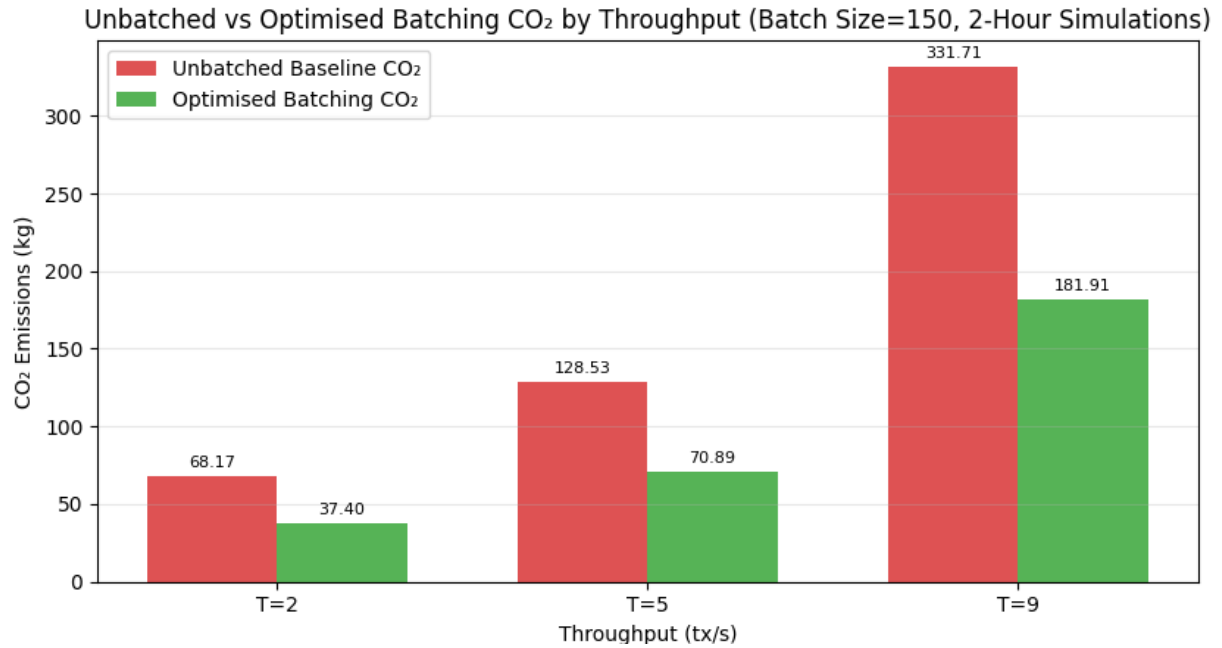
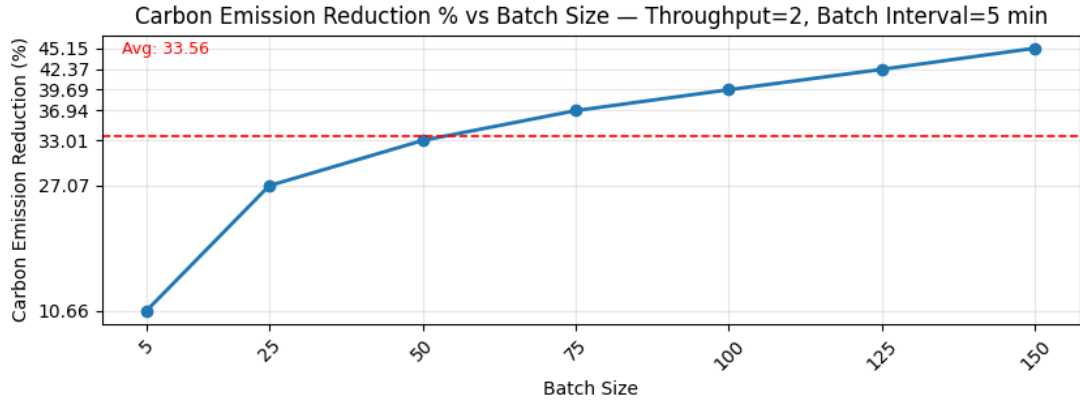
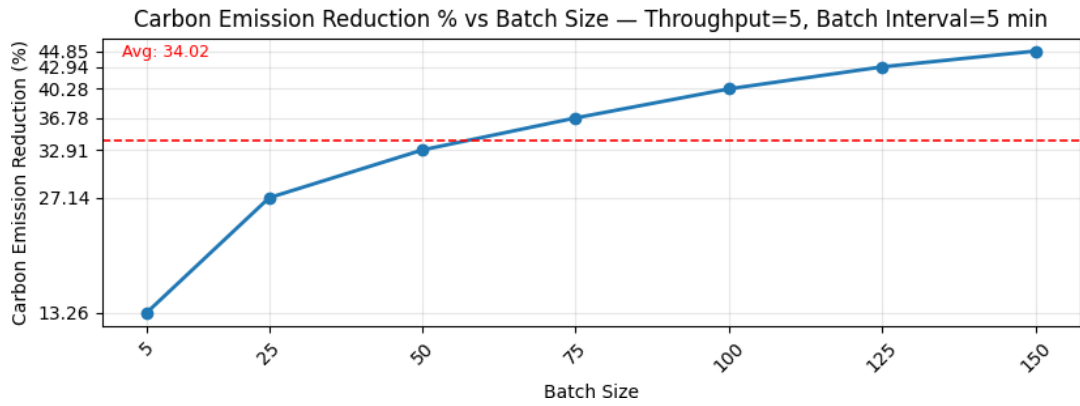


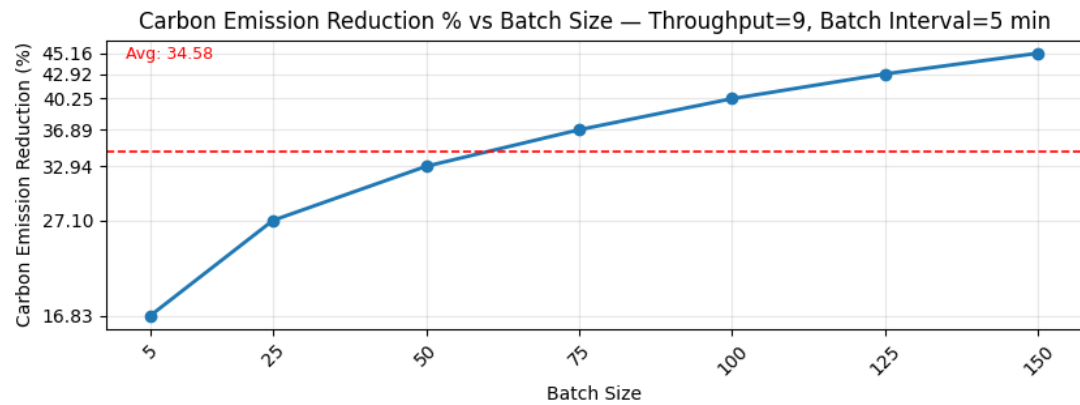
Figure C.3: Bar chart comparing unbatched and optimised batching CO₂ emissions at a batch size of 150 for the Ethereum network.



(a) Throughput = 2

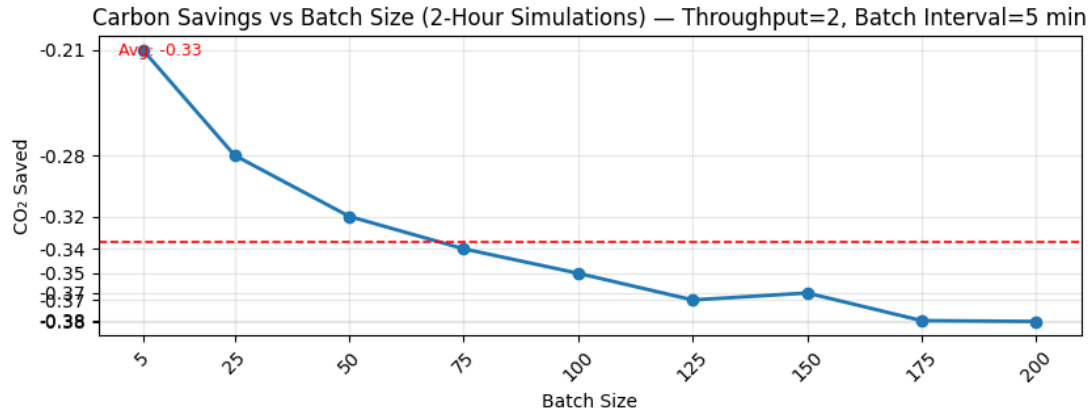


(b) Throughput = 5

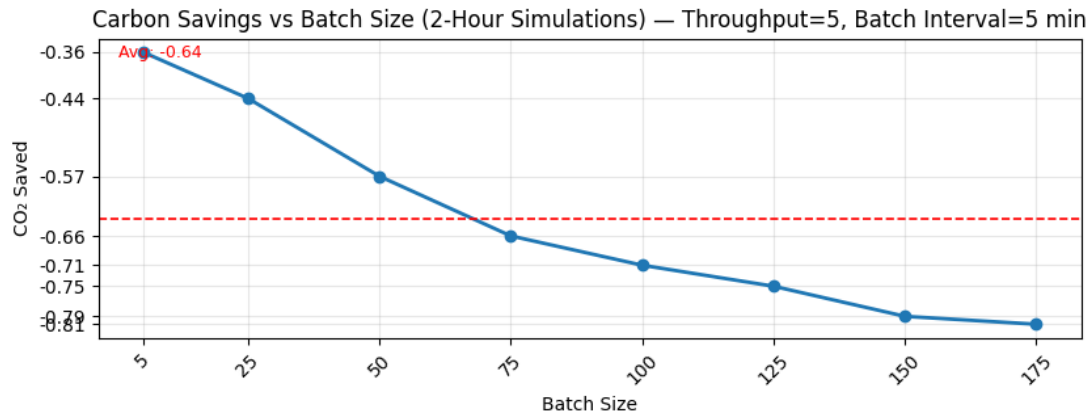


(c) Throughput = 9

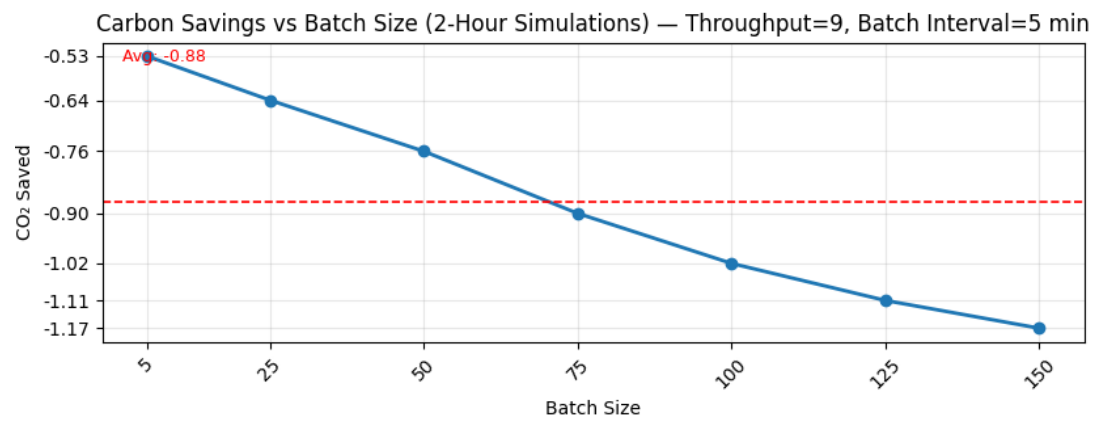
Figure C.4: Line graphs illustrating the relationship between carbon emission reduction percentage and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the Ethereum network.



(a) Throughput = 2



(b) Throughput = 5



(c) Throughput = 9

Figure C.5: Line graphs illustrating the relationship between absolute CO₂ savings and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

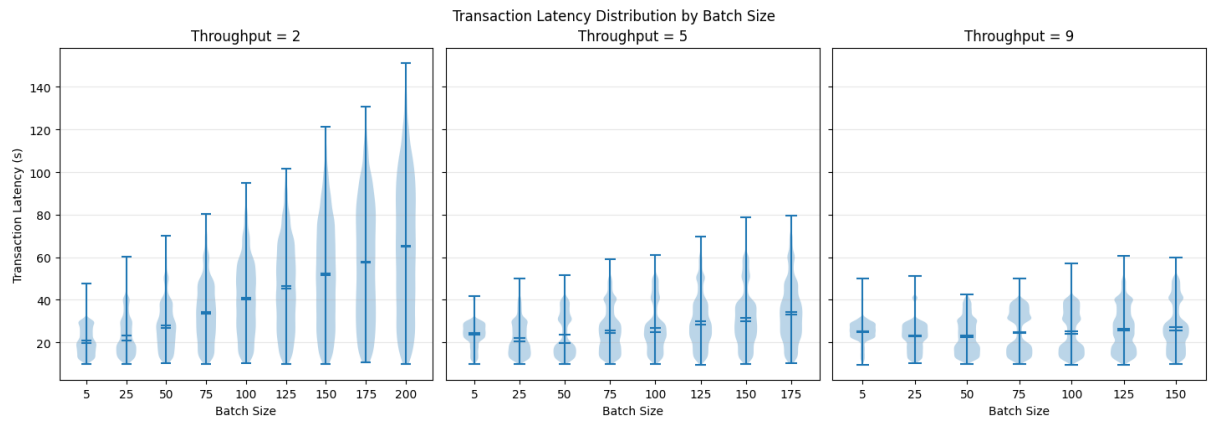


Figure C.6: Violin plots demonstrating the range and distribution of transaction latencies for throughput levels of 2, 5, and 9 TPS for the VeChain network.

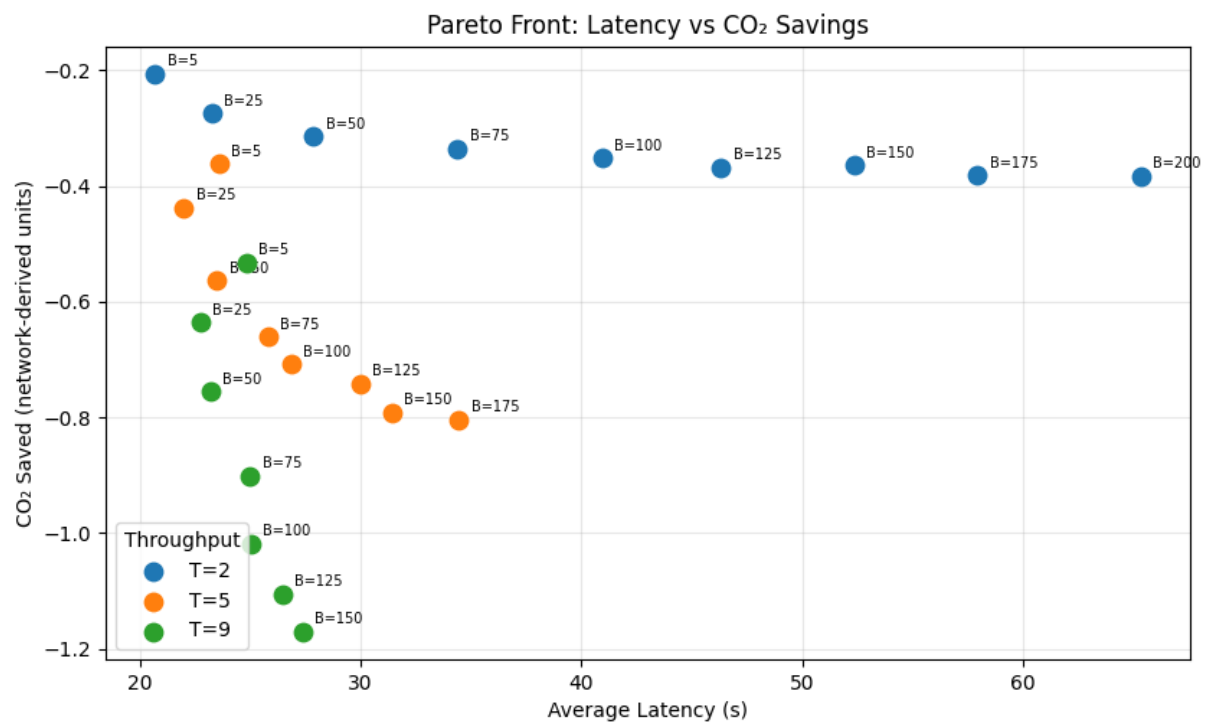
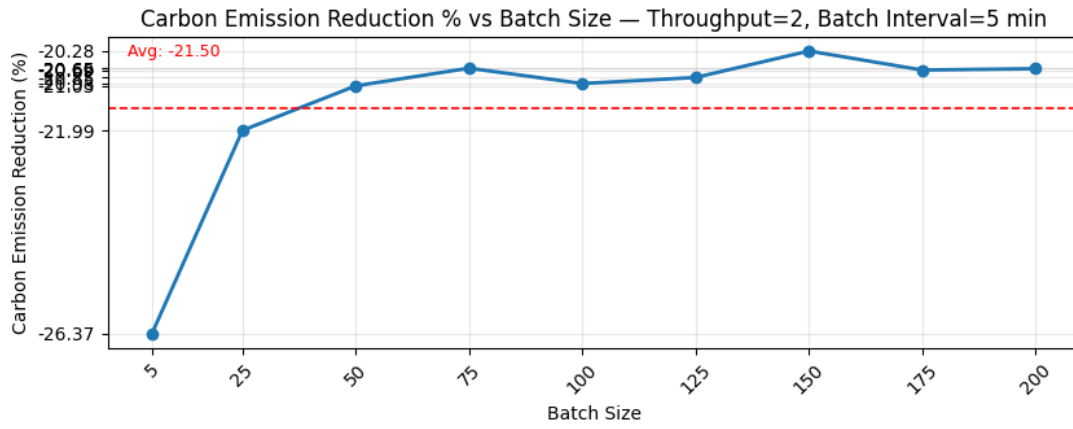
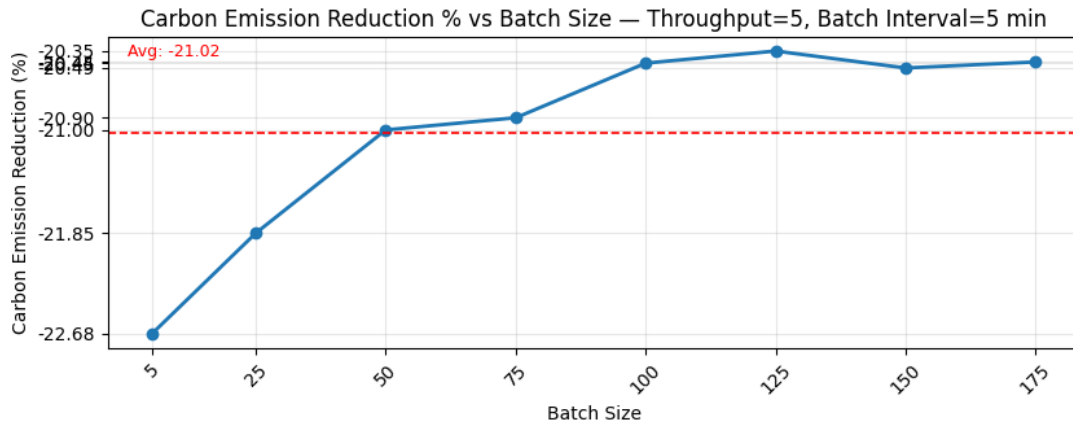


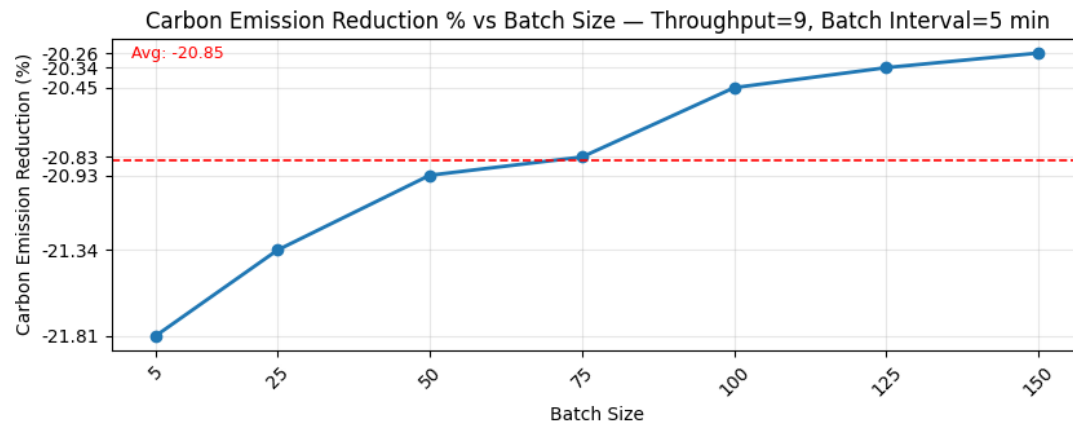
Figure C.7: Pareto front scatter plot showing the trade-off between average latency and absolute CO₂ savings for the VeChain network.



(a) Throughput = 2



(b) Throughput = 5



(c) Throughput = 9

Figure C.8: Line graphs illustrating the relationship between carbon emission reduction percentage and batch size across two-hour simulations at throughput levels of 2, 5, and 9 for the VeChain network.

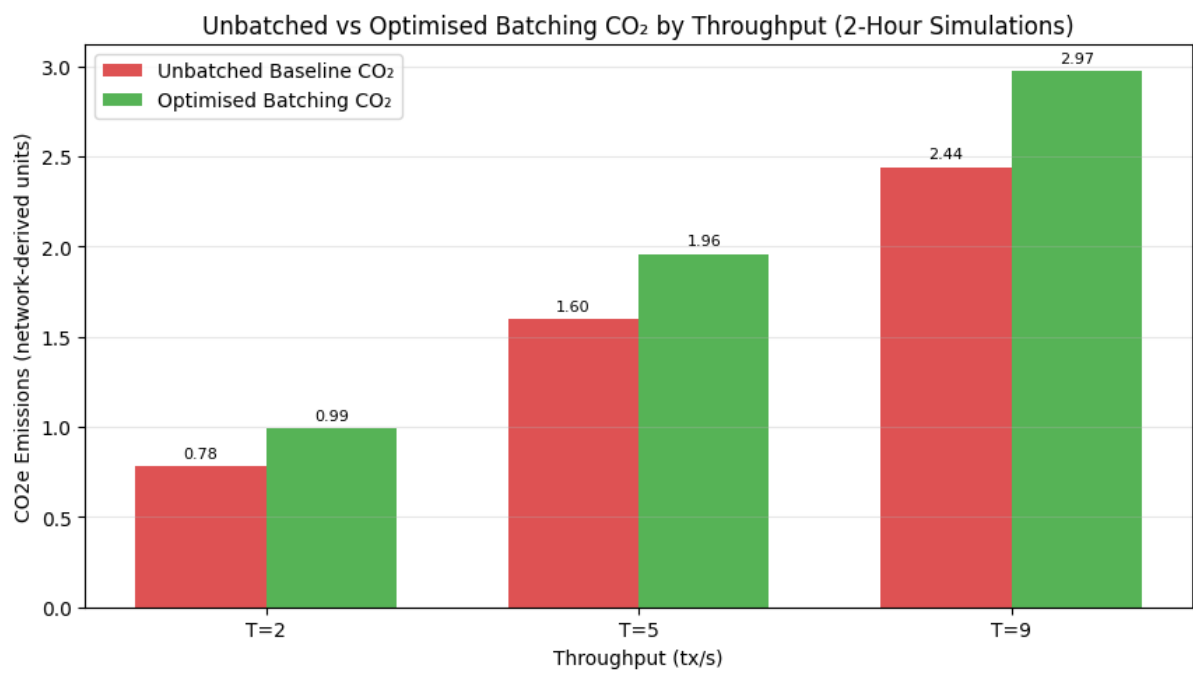


Figure C.9: Bar chart comparing unbatched baseline and optimised batching CO₂ emissions across throughput levels of 2, 5, and 9 for the VeChain network.